

테스트 자동화와 GitHub Actions

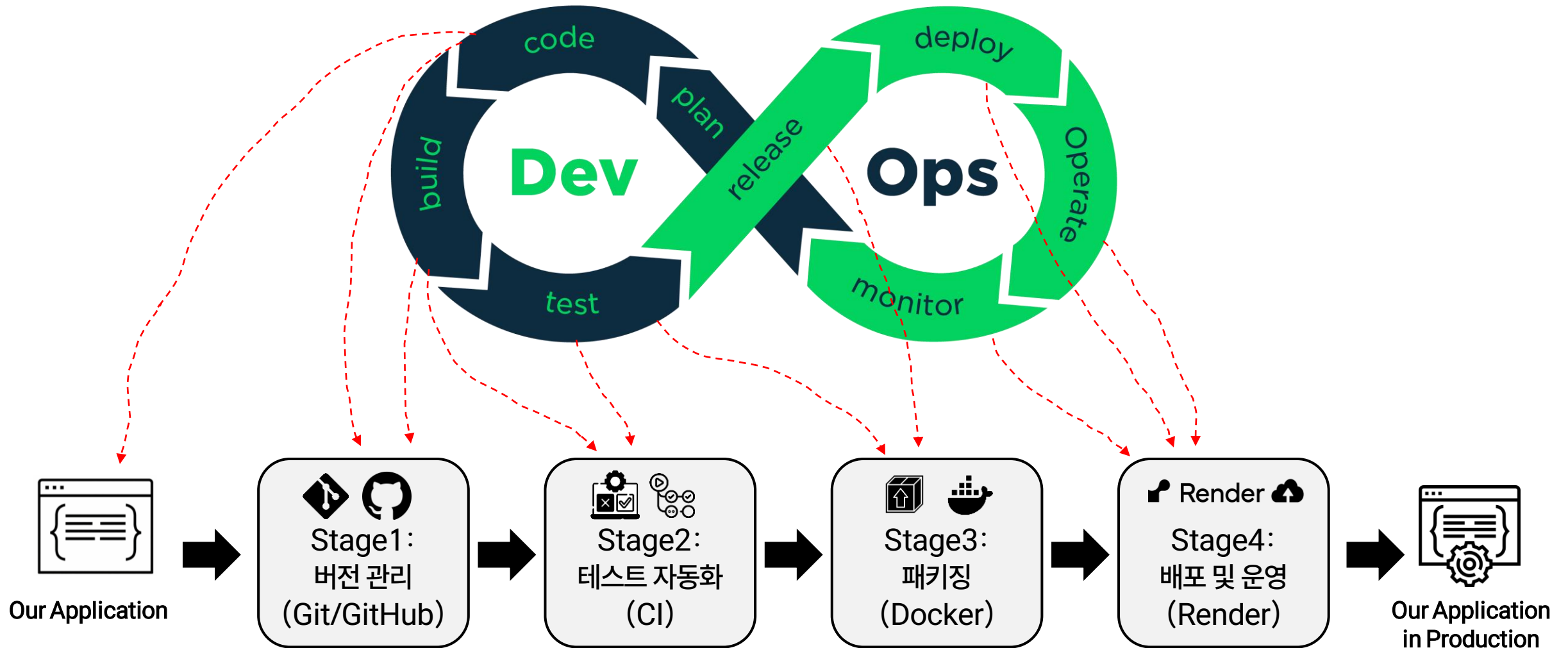
김미수

전남대학교 AI융합대학 인공지능학부

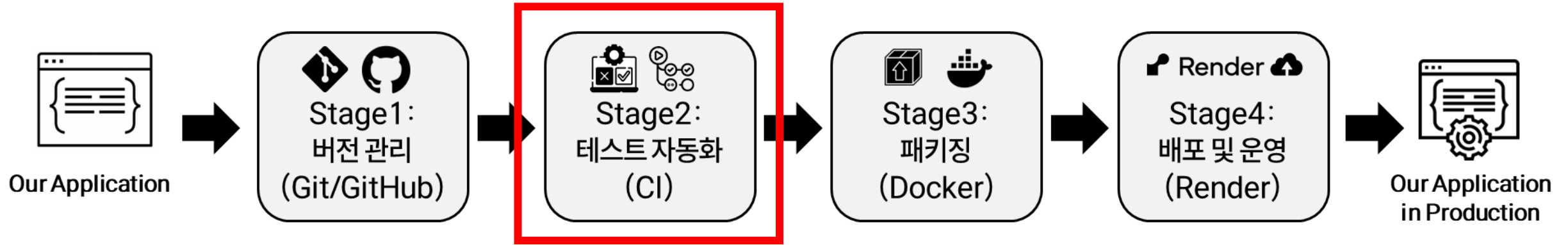
misoo.kim@jnu.ac.kr

<https://sites.google.com/view/semi-jnu>

수업에서 진행할 파이프라인



DevOps 파이프라인



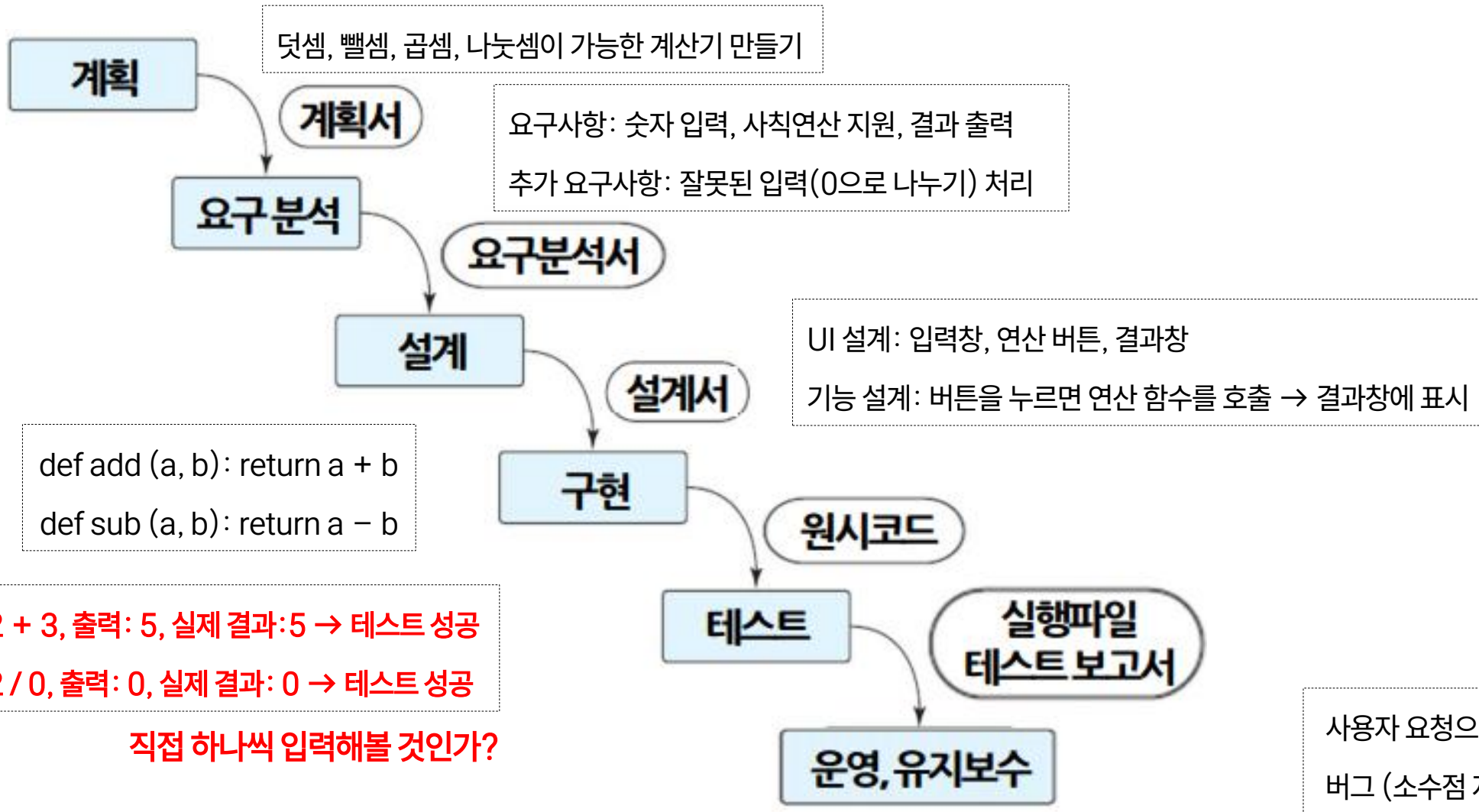
• 1단계: Git/GitHub & CI/테스트

- Git/GitHub: 로컬 코드를 팀과 공유하고, 변경 사항을 기록하고 관리
 - Git: 변경 기록, GitHub: 변경 사항의 원격 저장소
- CI: 코드 통합 시 **자동으로 테스트**해 빌드가 깨지는 것 방지
- 사용 Tool: GitHub Actions



소프트웨어 개발 절차

• 예시: 폭포수 모델



테스팅 용어 정리

Test Case(specification)

입력(a, b, c)	예상 출력
1, 4, 3	x1=-1, x2=-3
1, 2, 1	x1=1, x2=1
1, 2, 2	...
0, x, x	오류메시지
...	...

Test Suite

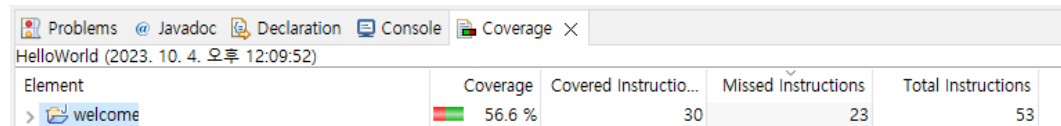
Test Data(real value)

테스팅 용어 정리

- 테스트 커버리지

- 소프트웨어에 대해 충분히 테스트 되었는가를 나타내는 척도
- 예: 문장 (statement, line) 커버리지

```
public static void main(String[] args) {  
    int month = 7;//월 데이터  
    String season;//계절 정보  
    if(month == 3 || month == 4 || month == 5) {  
        season = "봄";  
    }  
    else if(month == 6 || month == 7 || month == 8) {  
        season = "여름";  
    }  
    else if(month == 9 || month == 10 || month == 11) {  
        season = "가을";  
    }  
    else {  
        season = "겨울";  
    }  
    System.out.println("계절 = " + season);  
}
```



The screenshot shows an IDE's Coverage window for a file named 'HelloWorld'. The window has tabs for Problems, Javadoc, Declaration, Console, and Coverage. The Coverage tab is active, displaying a table with the following data:

Element	Coverage	Covered Instructio...	Missed Instructions	Total Instructions
> welcome	56.6 %	30	23	53

소프트웨어 동적 테스트

- 소프트웨어를 직접 실행시켜서 확인하는 작업
- 블랙박스 테스트: 내부 로직은 모르고 입력/출력만 확인
 - 사용자 입장에서 기능이 요구사항에 맞게 동작하는지 확인

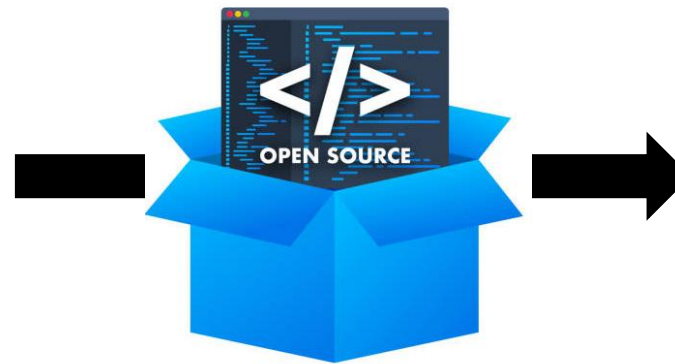
[요구사항]

회원 나이는 0~120 사이여야 한다.

테스트 케이스

테스트 입력	기대 결과
-100	에러 메시지
0	0
100	100
120	120
150	에러 메시지

테스트 스위트 (test suite)



실제 결과
-100
0
100
120
에러 메시지



디버깅 필요

소프트웨어 동적 테스트

- 소프트웨어를 직접 실행시켜서 확인하는 작업
- 화이트박스 테스트: 소스코드 경로를 고려하여 테스트
 - 주 목표: 코드의 모든 부분이 테스트 되는지 보장하기 → 내가 모든 기능을 한 번 이상 테스트 해봤나?

테스트 입력	기대 결과
10	Teen
50	Adult

```
def check_age(age):  
    if age < 0:  
        return "Invalid"  
    elif age <= 18:  
        return "Teen"  
    else:  
        return "Adult"
```

테스트 케이스 추가

테스트 입력	기대 결과
-1	Invalid

소프트웨어 테스트에서 중요한 점

- 어떤 입력을 넣을 것인가?
 - 문제: 넣어볼 수 있는 입력의 수는 무제한
 - 대표성 있는 입력
 - 버그를 일으킬 가능성이 높은 입력

일반적으로 테스트는 어떻게 할까?

• 테스트 시나리오

- 사용자가 올바른 아이디와 비밀번호를 입력하면

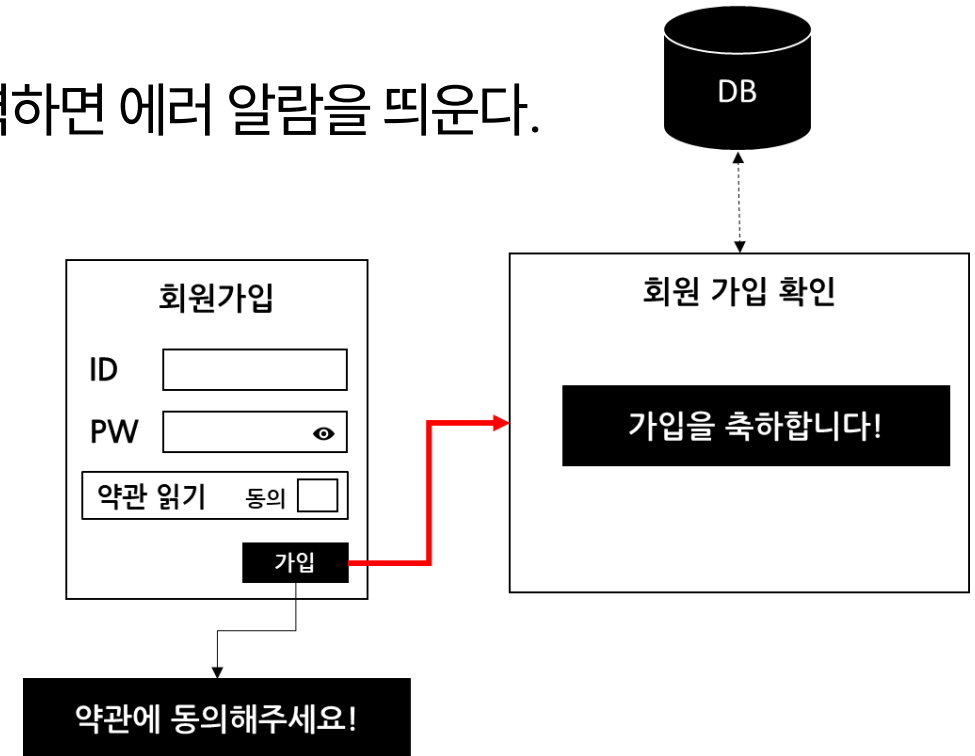
회원 가입 확인 페이지로 이동하고, 잘못된 비밀번호를 입력하면 에러 알람을 띄운다.

• 로그인 기능을 위한 테스트 케이스 (TC)

ID	ID	PW	약관동의	기대 결과
TC1	abc	1234567 (7자)	0	가입실패 알람

• 로그인 기능을 위한 테스트 스위트 (Suite)

ID	ID	PW	약관동의	기대 결과
TC-LG1	abc	1234567 (7자)	0	가입실패 알람
TC-LG2	user	abcdefgh (8자)	0	가입 성공 페이지
TC-LG3	xyz123	qwer12345 (9자)	0	가입 성공 페이지

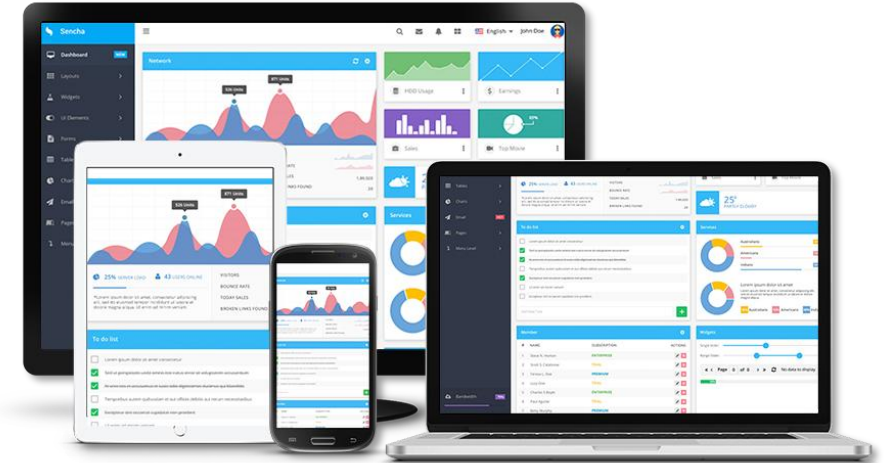


일반적으로 테스트는 어떻게 할까?

- 문서화된 테스트 스위트 → 테스터가 직접 확인해보기

ID	구분	Level 1	Level 2	Description
UT-02-0001	메일	초기화면	편지함 정보표시	메일 초기화면에서 편지함별 안읽은 메일 개수, 저장공간 View 기능
UT-02-0002	메일	기본기능	제목 작성	메일의 제목 작성 기능
UT-02-0003	메일		본문 작성	No Active-X를 사용하여 본문 작성
UT-02-0004	메일		파일첨부	메일 작성시 한 개의 메일 내용에 여러 개의 파일을 첨부하여 송신할 수 있게 하는 기능(Drag&Drop 포함) - 종류: 아스키 텍스트, 이진 파일, 이미지, 사운드, 동화상 이미지 등
UT-02-0005	메일	메일송신	발송메일 저장여부	수신자에게 발송한 메일 "보낸편지함" 저장여부 설정 기능
UT-02-0006	메일	메일수신	수신	다른 사람으로부터의 메일을 받는 기능
UT-02-0007	메일	메일 연속 열람	메일 연속 열람	이전/다음 메일을 연속해서 열람할 수 있는 기능
UT-02-0008	메일		수신 메일 열람	수신한 메일 목록을 확인하는 기능

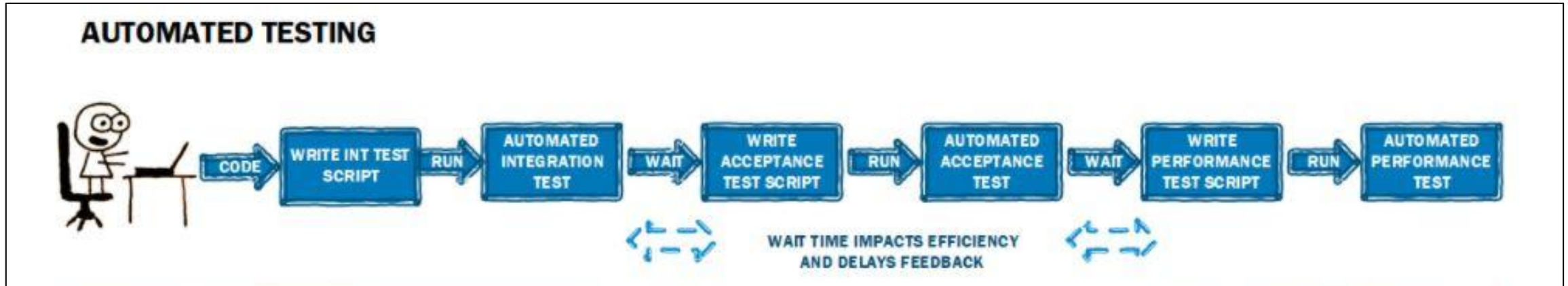
시스템명	스마트앱명	서버시스템명	행복포털			버전	v1.0
통합테스트 시나리오 ID	IT-SCO-001		통합테스트 시나리오명	행복포털 초기화면			
시나리오 설명	행복포털 초기화면 로그인 및 접속 정보 확인						
시험일자	업무내용	시험항목	사전조건	입력자료	예상결과	확인자	시험결과
전사행복포털 확인	초기화면	전사행복포털	정규적으로 로그인	1) www.happyph.co.kr에 크롬 브라우저로 접속 2) 인증서로그인 클릭 후 PC의 PKI인증서 선택 3) 인증서 위치: 하드디스크 4) 인증서를 마우스로 선택 후 인증서 암호 입력 후 확인 버튼 클릭 5) 전사행복포털 확인	화면 상단에 메뉴가 나타나고 메인 영역에 포틀릿이 나타남 상단: 메인메뉴, 알림, 직원검색, 행복포털설정, 통합검색 포틀릿: 공지사항, 협업마당, 복리후생, 임원일정 등		
내부 행복포털 확인	초기화면	내부 행복포털 설정	정규적으로 로그인	1) 전사행복포털 초기화면에서 행복포털설정 클릭 2) 내부행복포털 버튼 클릭 3) 닫기 버튼 클릭	화면 상단의 내용은 전사행복포털과 동일하고 텍스트 위주의 포틀릿이 나타남 포틀릿: 행복한 사람들 소식, 공지사항, 홍보소식, 자유정보, 공지사항, 해외행복한사람들 후무보기 등		
업무행복포털 확인	업무행복포털	업무행복포털 확인	정규적으로 로그인	1) 로그인 후 전사행복포털에서 상단의 스마트앱업 메뉴 클릭	화면 상단의 내용은 전사행복포털과 동일하고 메인 영역에 스마트앱업 업무 관련 포틀릿이 나타남 포틀릿: 업무카운트(메일, 출재, 게시판 등), 나의 할일, 출재, 메일, 알림, 사물관리 등		
강한 그룹 별 행복포털 확인	초기화면	전사행복포털	인턴으로 로그인	1) 로그인 후 전사행복포털에서 상단의 스마트앱업 메뉴 클릭	화면 상단에 메뉴가 나타나고 메인 영역에 포틀릿이 나타남 상단: 메인메뉴, 알림, 직원검색, 행복포털설정, 통합검색 포틀릿: 공지사항, 복리후생 등		
통합 알림 확인	초기화면	알림 확인	정규적으로 로그인	1) [내부 우측의 홈 아이콘 클릭	출재, 메일, 게시판 등의 알림을 한번에 확인		
통합 검색	초기화면	검색엔진 연계		1) 행복포털 로그인 2) 화면 우측 상단의 검색어 칸에 검색어 '여행' 입력	여행 관련 내용이 출재, 게시판 등에서 검색되어 팝업으로 나타난다.		
인사 시스템 연계	초기화면	인사출재 확인		1) 로그인 후 전사행복포털에서 상단의 HAPPY2O 메뉴 클릭 2) 화면 좌측의 인사출재 카운트 확인 3) 카운트 숫자 클릭	인사시스템의 출재전수와 내용을 확인한다.		
사장님 활동정보	초기화면	사장님 접속정보 확인		1) 로그인 후 전사행복포털에서 메뉴의 사장님 메뉴 클릭 2) 화면 좌측의 회의/출장 현황 확인 3) 카운트 숫자 클릭	사장님 일정에서 해당 일의 카운트 숫자를 클릭하면 좌측 화면의 상세 내역(회의 및 출장 등)이 출력된다.		



- 문제가 생겨서 코드를 고친 후라면... 매번 다 손으로 실행하고 눈으로 확인해야 할까?

테스트 자동화

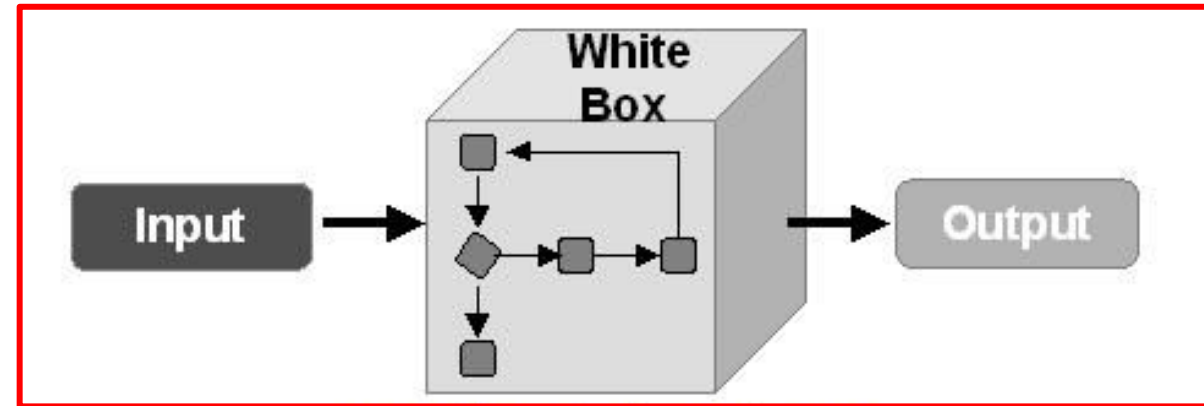
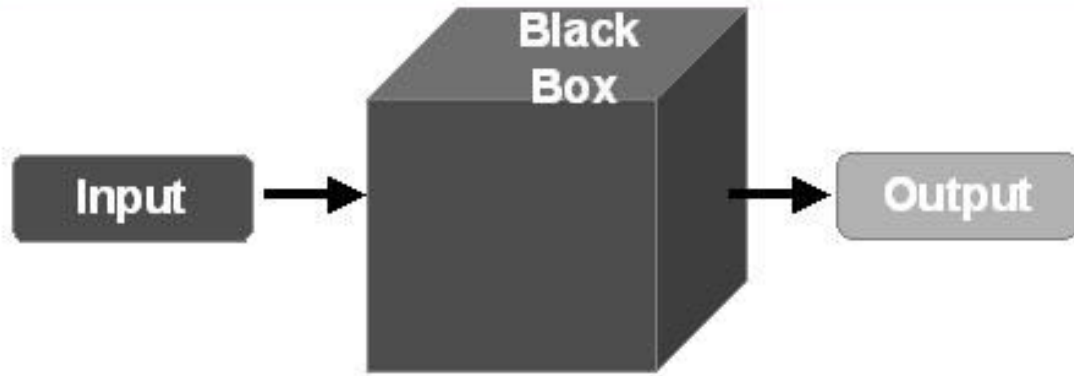
- 도구나 코드 (테스트 스크립트)를 사용해 테스트를 자동화 하는 것
 - 단계: 테스트 코드 작성 → 자동으로 실행 및 관리



- 개발자 입장에서 충분한 테스트 코드를 만들어서, 기능적인 부분을 확인하는 것을 우선
 - 사용자 입장이 고려되어야 하면 테스트 시나리오를 기반으로 테스터가 직접 진행할 수 있음

화이트박스 테스트 – 구조 기반 테스트 설계

- 소스 코드 구조를 기반으로 테스트 케이스를 설계



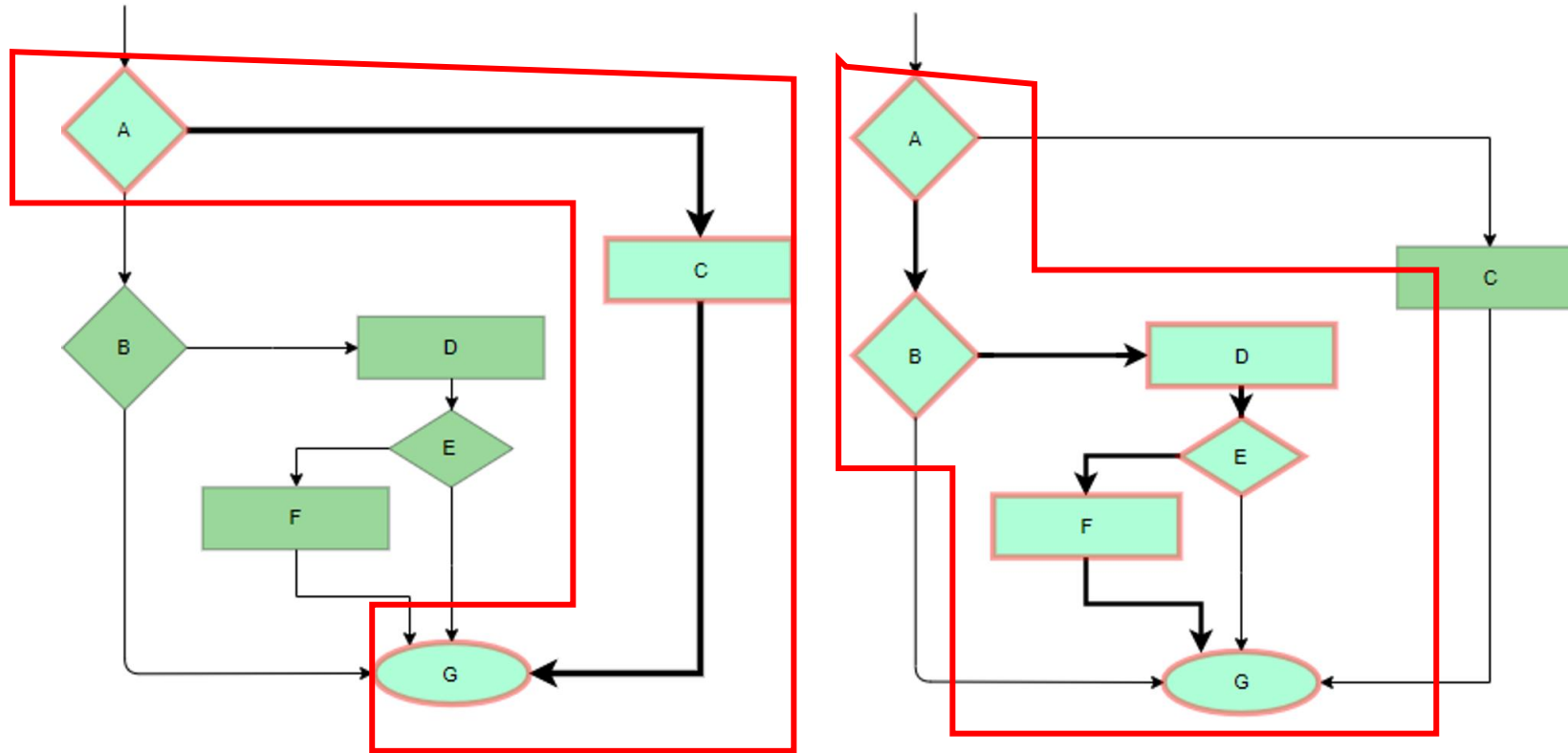
- 주 목표: 문장 커버리지 (Statement Coverage) 최대화

$$= (\text{수행된 문장의 수} / \text{전체 문장의 수}) * 100$$

- 경로 기반 테스트 케이스를 선정하는 첫 번째 방법
- 코드를 구성하는 모든 문장을 최소한 한 번씩은 거쳐 가야 한다는 조건을 만족하도록 패스 선택
- 코드를 구성하는 모든 문장을 한 번씩 거쳐 가면 문장 커버리지를 100% 달성하는 테스트 스위트를 구성하는 것이 목표

문장 커버리지 기반 생성

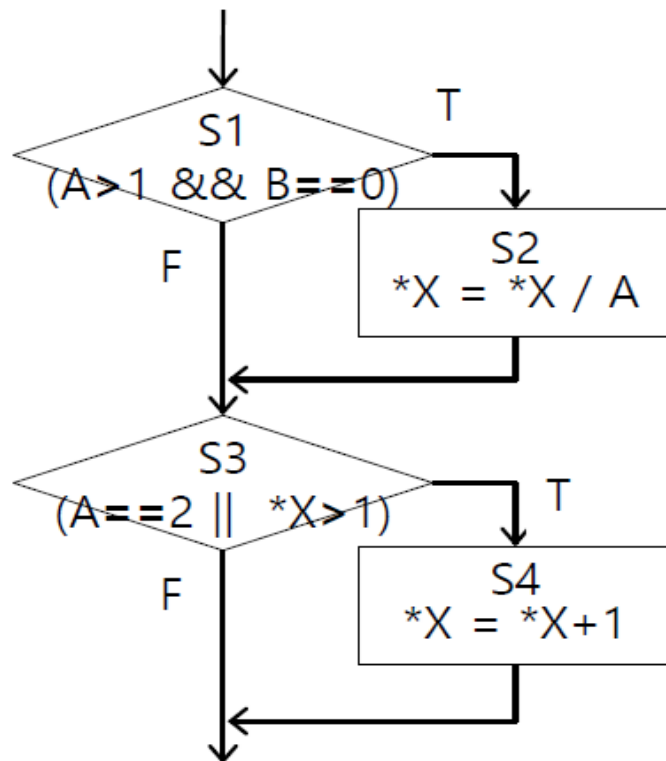
- 문장 (statement) 커버리지 기반 = 최소 2개의 테스트 케이스 생성



문장 커버리지 기반 생성

• 예제. 문장 커버리지

	TC1			TC2		
	A	B	*X	A	B	*X
	3	0	1	2	0	3
S1	√			√		
S2	√			√		
S3	√			√		
S4				√		
	3 / 4			4 / 4		
	4 / 4					



```

void process(int A, int B, int* X) {
    // S1: 조건 (A > 1 && B == 0)
    if (A > 1 && B == 0) {
        // S2: *X = *X / A
        *X = *X / A;
    }

    // S3: 조건 (A == 2 || *X > 1)
    if (A == 2 || *X > 1) {
        // S4: *X = *X + 1
        *X = *X + 1;
    }
}
  
```

TC2만 있어도 문장 커버리지는 100%

분기 커버리지 기반 생성

- 분기 커버리지 (Branch Coverage)

$$= (\text{수행된 분기의 수} / \text{전체 분기의 수}) * 100$$

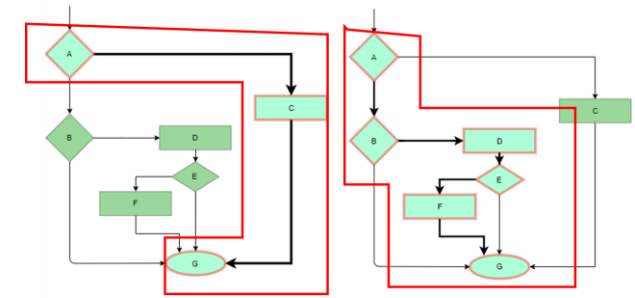
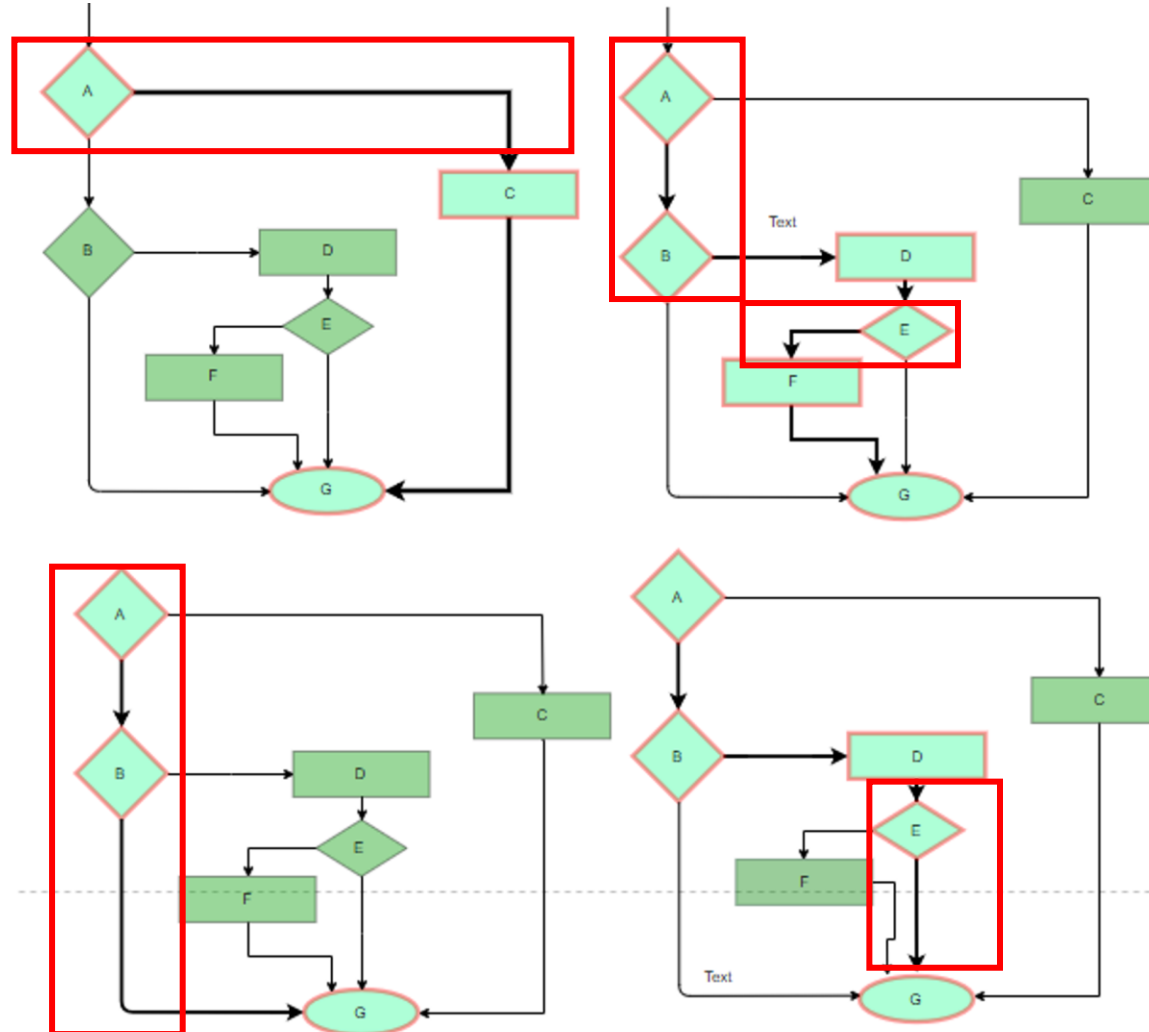
- 결정 커버리지 (decision coverage) 라고도 함
- 모든 분기를 적어도 한 번씩은 거쳐 가도록 패스를 선정하는 방법
 - if 문의 경우 참과 거짓의 두 경로가 각각 한 번씩 선정되어야 함
 - switch 문에서는 모든 case 문과 default 문이 선정되어야 함
 - for/while 문에서는 적어도 한 번 루프의 내부가 실행되어야 함
- 100% 문장 커버리지 수행이 모든 분기를 포함하는 것은 아님

A	B	A OR B
True	True	True
True	False	True
False	True	True
False	False	False

프로그램 내 전체 분기가 적어도 한번은 참과 거짓의 결과를 수행하도록 설계

분기 커버리지 기반 생성

- 분기 커버리지 (Branch Coverage) : 최소 4개의 TC 필요



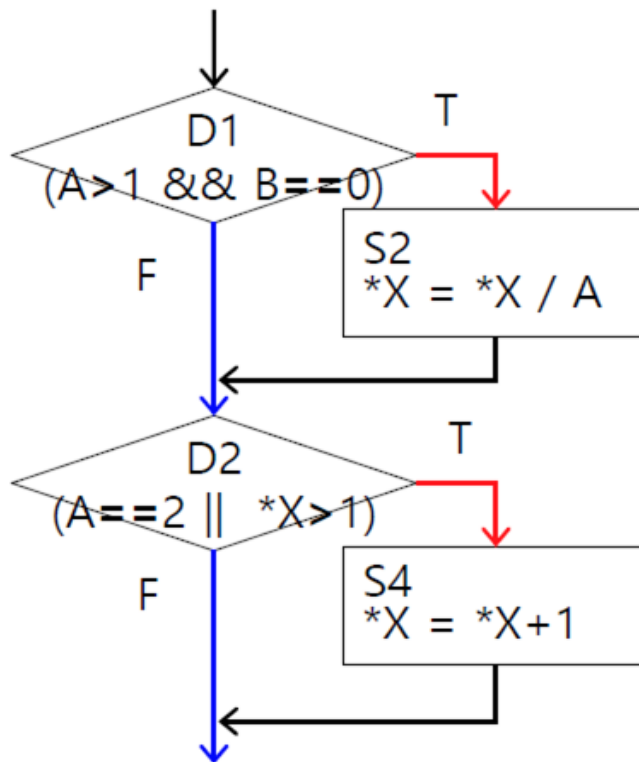
<문장커버리지 100% TC>

분기 커버리지 기반 생성

• 예제. 분기 커버리지

		TC1	TC1
		A=2 B=0 *X=4	A=1 B=1 *X=1
D1	A > 1	T	F
	B == 0	T	F
D2	A == 2	T	F
	*X > 1	T	F
		2 / 4	2 / 4
		4 / 4	

D: decision (=branch)



```

void process(int A, int B, int* X) {
    // S1: 조건 (A > 1 && B == 0)
    if (A > 1 && B == 0) {
        // S2: *X = *X / A
        *X = *X / A;
    }

    // S3: 조건 (A == 2 || *X > 1)
    if (A == 2 || *X > 1) {
        // S4: *X = *X + 1
        *X = *X + 1;
    }
}
  
```

조건 커버리지 기반 생성

- 조건 커버리지 (Condition Coverage)

$$= (\text{수행된 조건의 수} / \text{전체 조건의 수}) * 100$$

- 조건문에 대하여 모든 가능한 결과가 적어도 한 번씩은 나타날 수 있도록 데이터 케이스를 준비하는 방법
 - 각 조건식의 True, False를 모두 실행할 수 있도록 하기
- 진리표에 나타날 수 있는 모든 경우에 대하여 테스트 케이스를 준비
 - 모든 명제와 조합에 대한 입출력 결과인 진릿값을 기록한 표

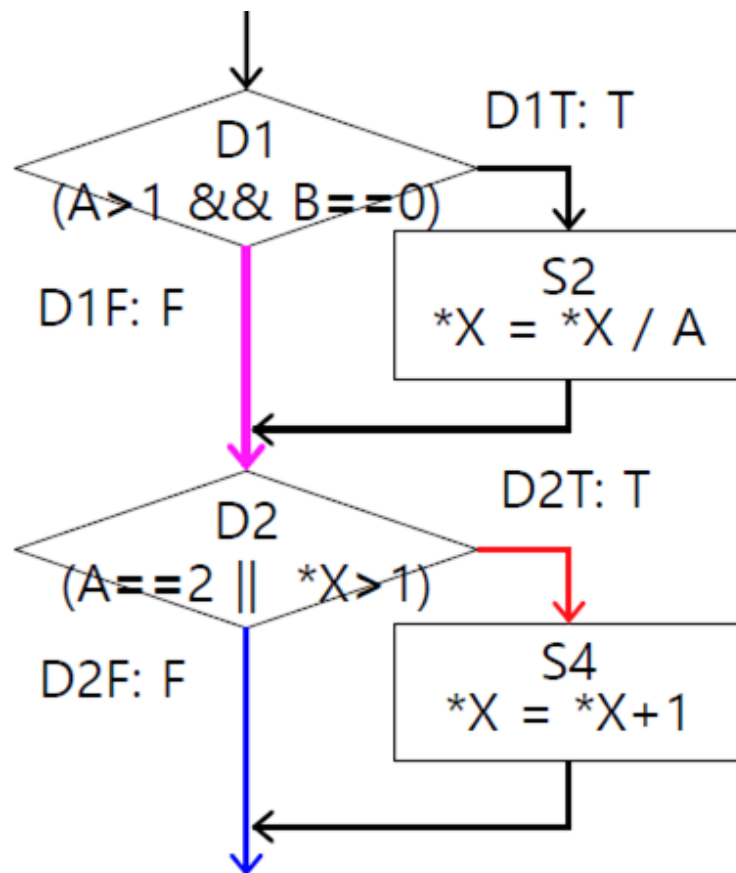
A	B	A OR B
True	True	True
True	False	True
False	True	True
False	False	False

분기 내 각 개별 조건이 적어도 한번은 참과 거짓의 결과가 출력되도록 수행

조건 커버리지 기반 생성

• 예제. 조건 커버리지

	TC1	TC1
	A=1 B=0 *X=3	A=2 B=1 *X=1
A > 1	F	T
B == 0	T	F
A == 2	F	T
*X > 1	T	F
	4 / 8	4 / 8
	8 / 8	



```

void process(int A, int B, int* X) {
    // S1: 조건 (A > 1 && B == 0)
    if (A > 1 && B == 0) {
        // S2: *X = *X / A
        *X = *X / A;
    }

    // S3: 조건 (A == 2 || *X > 1)
    if (A == 2 || *X > 1) {
        // S4: *X = *X + 1
        *X = *X + 1;
    }
}
  
```

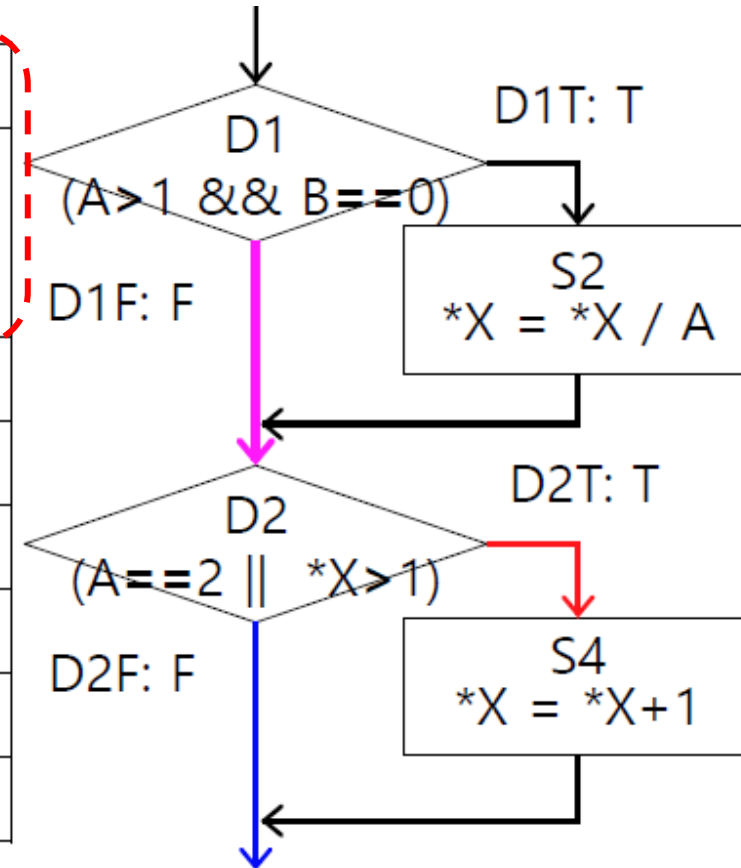
조건 커버리지 vs 분기 커버리지

동일한 테스트케이스

	TC1	TC1		TC1	TC2
	A=1 B=0 *X=3	A=2 B=1 *X=1		A=1 B=0 *X=3	A=2 B=1 *X=1
A > 1	F	T	D1T		
B == 0	T	F	D1F	✓	✓
A == 2	F	T	D2T	✓	✓
*X > 1	T	F	D2F		
	4 / 8	4 / 8		2 / 4	2 / 4
	8 / 8			2 / 4	

Condition cov. 만족.

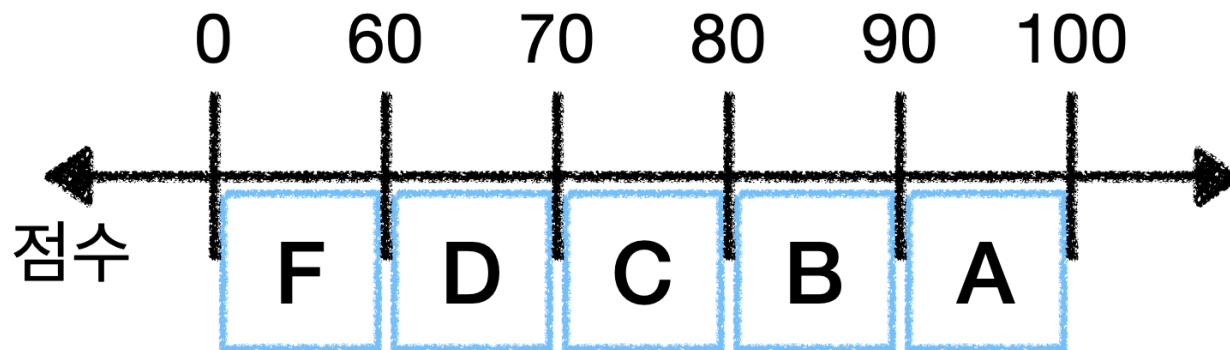
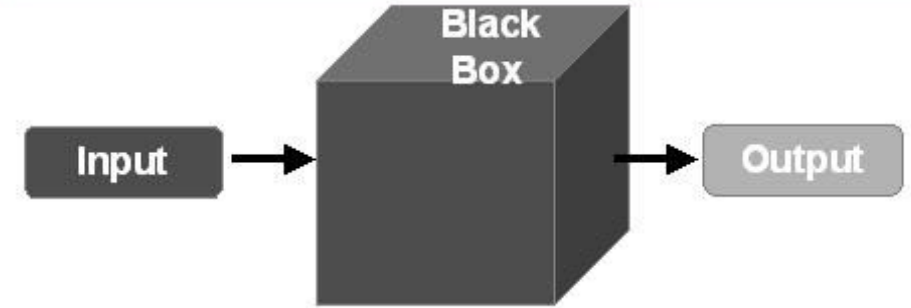
Branch cov. 만족하지 않음.



[참고: 채흥석 교수 강의]

블랙 박스 테스트 – 명세 기반 테스트 설계

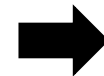
- 소스 코드 구조를 기반으로 테스트 케이스를 설계
- 주 방법 : 동등 (동치) 분할 기반 테스트 케이스 생성
 - 입력의 결과로 그룹을 나누어 대표 값을 선택
 - 프로그램 입력 데이터를 동일한 동작/결과가 예상되는 동등 클래스로 분할하고, 각 클래스의 대표값 (주로 중앙값) 으로 테스트 케이스를 도출하는 기법
 - 주로 명세서에 있는 입/출력 값을 기반으로 함



<https://codezeroman.tistory.com/36>

BRUTE FORCE (전체 탐색)

0 → F
1 → F
.....
99 → A
100 → 100



동등분할

30 → F
65 → D
75 → C
85 → B
95 → A

동등 분할 기반 테스트 케이스 생성

- 예시. 프로젝트 기참여율에 따라 신규 프로젝트 참여 여부를 결정하는 기능
 - 50% 이하이거나 90% 이상인 경우는 참여 불가
 - 51% ~ 89% 사이만 참여 가능

Percentage * Accepts Percentage value between 50 to 90

입력에 따른 결과 입력 그룹 설계한 테스트 입력 (테스트)	Equivalence Partitioning		
	Invalid	Valid	Invalid
	≤ 50	50-90	≥ 90
	30	70	95

<https://www.geeksforgeeks.org/equivalence-partitioning-method/>

경계값 기반 테스트 케이스 생성

- 동등 분할 후 클래스 **경계값**을 테스트 데이터로 선정

- 입력 변수에 정의된 값의 특정 영역에 대한 경계값을 선정
- 주로 $min - 1, min, max, max + 1$ 로 선정

- 예1: 입력 변수 = $[-1.0, +1.0]$

▶ $TC = \{-1.1, -1.0, +1.0, +1.1\}$

- 예2: 입력 변수 = $1 \sim 100$

▶ $TC = \{0, 1, 100, 101\}$

- 예3: 입력 순서 집합 = $\{A, B, C, D, F\}$

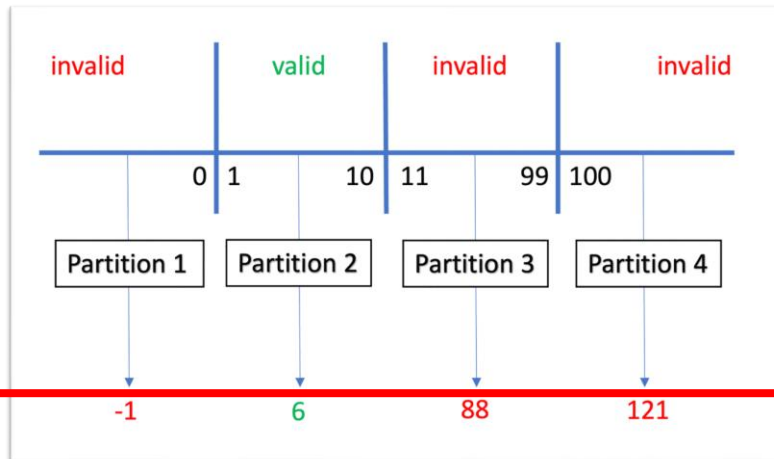
▶ $TC = \{A, F\}$

동등 분할 vs 경계값 분석

• 예시. 온라인 티켓 예약

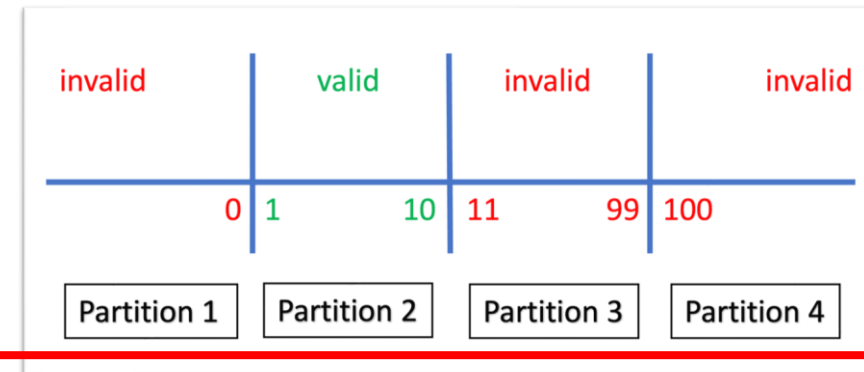
1~10개	"성공"
11~99개	"최대 10개까지 가능"
> 99	"오류"
< 1	"1개 이상을 선택하세요"

동치 분할



Test Case	Number of Tickets	Expected Output
TC1	-1	"You must select at least one ticket" message
TC2	6	Success message
TC3	88	"Maximum 10 tickets can be reserved." message
TC4	121	"General system error". message

경계치 커버리지



Test Case	Number of Tickets	Expected Output
TC1	0	"You must select at least one ticket" message
TC2	1	Success message
TC3	10	Success message
TC4	11	"Maximum 10 tickets can be reserved." message
TC5	99	"Maximum 10 tickets can be reserved." message
TC6	100	"General system error." message

효율적이나, 정확도 한계



전남대학교

보다 정확하나, TC 수 증가

예제 - 테스트 설계

- 요구사항: 성적 등급 산출

- 90점 이상 A, 60점이상 90점 미만 B, 그 외 F
- 점수는 0~100 사이의 정수여야 함

- 문장 커버리지 기반

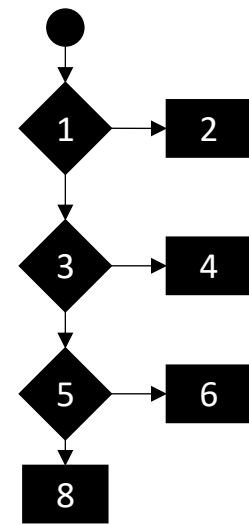
입력	기대 결과	지나간 라인 (커버리지)
-1	Invalid	1, 2
95	A	1-4
65	B	1-6
50	F	1-8

A	B	A OR B
True	True	True
True	False	True
False	True	True
False	False	False

- 분기 커버리지

입력	기대 결과	분기 커버리지		
		1	3	5
-1	Invalid	TRUE		
95	A	FALSE	TRUE	
65	B	FALSE	FALSE	TRUE
50	F	FALSE	FALSE	FALSE

```
def grade(score: int) -> str:
1  if score < 0 or score > 100:
2      return "Invalid"
3  elif score >= 90:
4      return "A"
5  elif score >= 60:
6      return "B"
7  else:
8      return "F"
```



A	B	A OR B
True	True	True
True	False	True
False	True	True
False	False	False

- 조건 커버리지 기반

- 조건/분기, 다중조건 모두 만족

입력	기대 결과	조건 커버리지			
		1.1	1.2	3	5
-1	Invalid	TRUE			
101	Invalid	FALSE	TRUE		
95	A	FALSE	FALSE	TRUE	
70	B	FALSE	FALSE	FALSE	TRUE
50	F	FALSE	FALSE	FALSE	FALSE

예제 – 테스트 설계

• 분기 커버리지

입력	기대 결과	분기 커버리지		
		1	3	5
-1	Invalid	TRUE		
95	A	FALSE	TRUE	
65	B	FALSE	FALSE	TRUE
50	F	FALSE	FALSE	FALSE

• 테스트 코드

```
import unittest
from sample import *

class TestGradeConditionCoverage(unittest.TestCase):
    def test_negative_score(self):
        self.assertEqual(grade(-1), "Invalid")

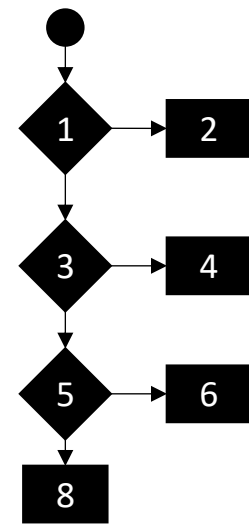
    def test_high_score(self):
        self.assertEqual(grade(95), "A")

    def test_mid_score(self):
        self.assertEqual(grade(70), "B")

    def test_low_score(self):
        self.assertEqual(grade(50), "F")

if __name__ == "__main__":
    unittest.main()
```

```
def grade(score: int) -> str:
1  if score < 0 or score > 100:
2      return "Invalid"
3  elif score >= 90:
4      return "A"
5  elif score >= 60:
6      return "B"
7  else:
8      return "F"
```



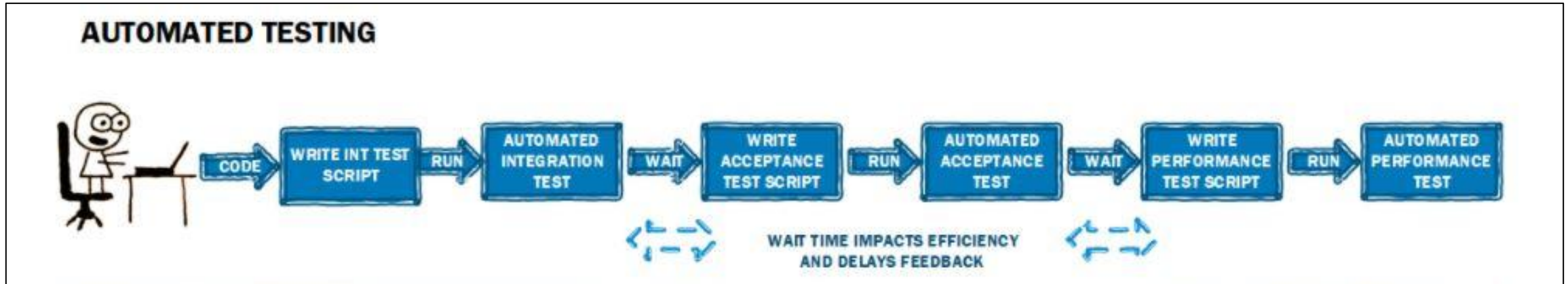
• 커버리지 확인

- coverage run --branch -m unittest discover

Name	Stmts	Miss	Branch	BrPart	Cover
sample.py	8	0	6	0	100%
test.py	13	1	2	1	87%
TOTAL	21	1	8	1	93%

테스트 자동화

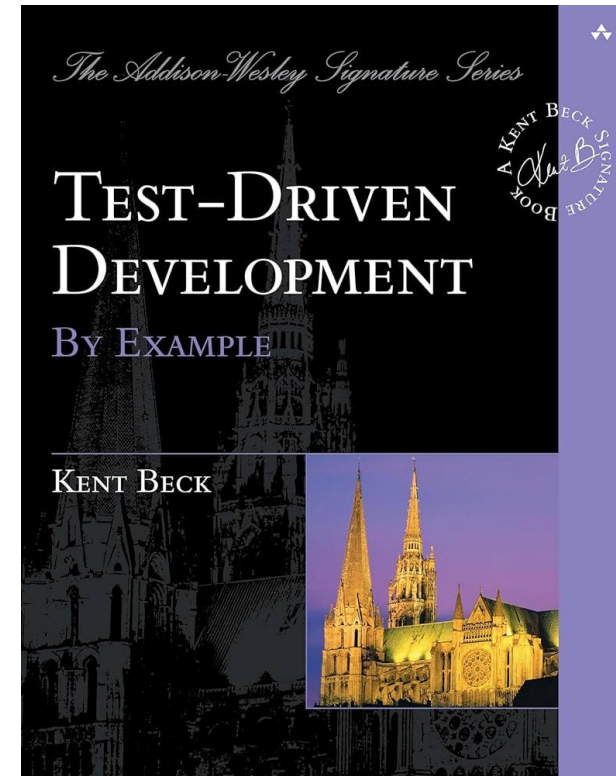
- 도구나 코드 (테스트 스크립트)를 사용해 테스트를 자동화 하는 것
 - 단계: 테스트 코드 작성 → 자동으로 실행 및 관리



테스트 주도 개발

- TDD: Test driven development

- 테스트를 먼저 작성하고, 테스트를 통과하는 최소한의 코드를 작성한 뒤, 리팩토링을 수행하는 개발 방식
- 목표: 결함이 적고 유지보수가 쉬운 코드를 작성
- “If it’s not tested, it’s broken.”

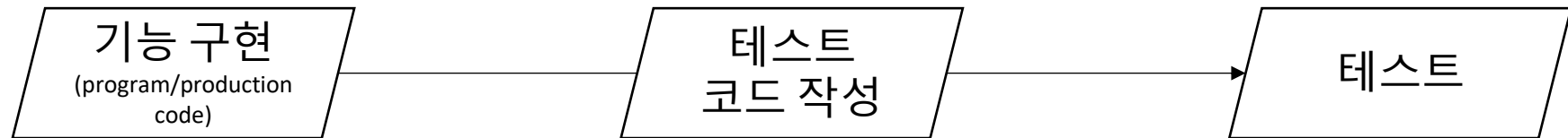


테스트 주도 개발

- TDD: Test driven development

- 테스트를 먼저 작성하고, 테스트를 통과하는 최소한의 코드를 작성한 뒤, 리팩토링을 수행하는 개발 방식
- 목표: 결함이 적고 유지보수가 쉬운 코드를 작성
- “If it’s not tested, it’s broken.”

일반적 절차



TDD



TDD Cycle

- Red: 실패하는 테스트 작성
- Green: 테스트를 통과시키는 최소한의 코드 작성
- Refactor: 중복 제거, 구조 개선



TDD Cycle

- Red: 실패하는 테스트 작성
- Green: 테스트를 통과시키는 최소한의 코드 작성
- Refactor: 중복 제거, 구조 개선, 기능 개선, 테스트 추가 등
- 간단한 예시 코드

1) TC 생성

```
1 # test_number_utils.py
2 import unittest
3
4 class TestIsEven(unittest.TestCase):
5     def test_even_number(self):
6         self.assertTrue(is_even(4))
7
8 if __name__ == '__main__':
9     unittest.main()
```

```
E
-----
ERROR: test_even_number (__main__.TestIsEven)
-----
Traceback (most recent call last):
  File "main.py", line 6, in test_even_number
    self.assertTrue(is_even(4))
NameError: name 'is_even' is not defined
-----
Ran 1 test in 0.000s
FAILED (errors=1)
```

2) PC 생성

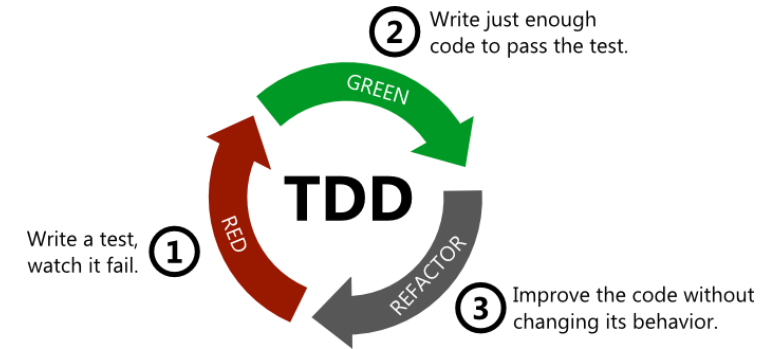
```
1 # number_utils.py
2 def is_even(n):
3     return n % 2 == 0
```

```
.
-----
Ran 1 test in 0.000s
OK
```

3.1) 기능 개선

```
1 # number_utils.py
2 def is_even(n):
3     def is_even(n):
4         if not isinstance(n, int):
5             raise ValueError("정수만 입력하세요")
6         return n % 2 == 0
```

```
.
-----
Ran 1 test in 0.002s
OK
```



3.2) 테스트 추가

```
7 # test_number_utils.py
8 import unittest
9
10 class TestIsEven(unittest.TestCase):
11     def test_even_number(self):
12         self.assertTrue(is_even(4))
13     def test_odd_number(self):
14         self.assertFalse(is_even(3))
15
16     def test_zero(self):
17         self.assertTrue(is_even(0))
18
19     def test_negative_even(self):
20         self.assertTrue(is_even(-2))
21
22     def test_non_integer(self):
23         with self.assertRaises(ValueError):
24             is_even("two")
25
26 if __name__ == '__main__':
27     unittest.main()
```

```
.
.....
-----
Ran 5 tests in 0.000s
OK
```


그 외에도...

• 뮤테이션 테스트

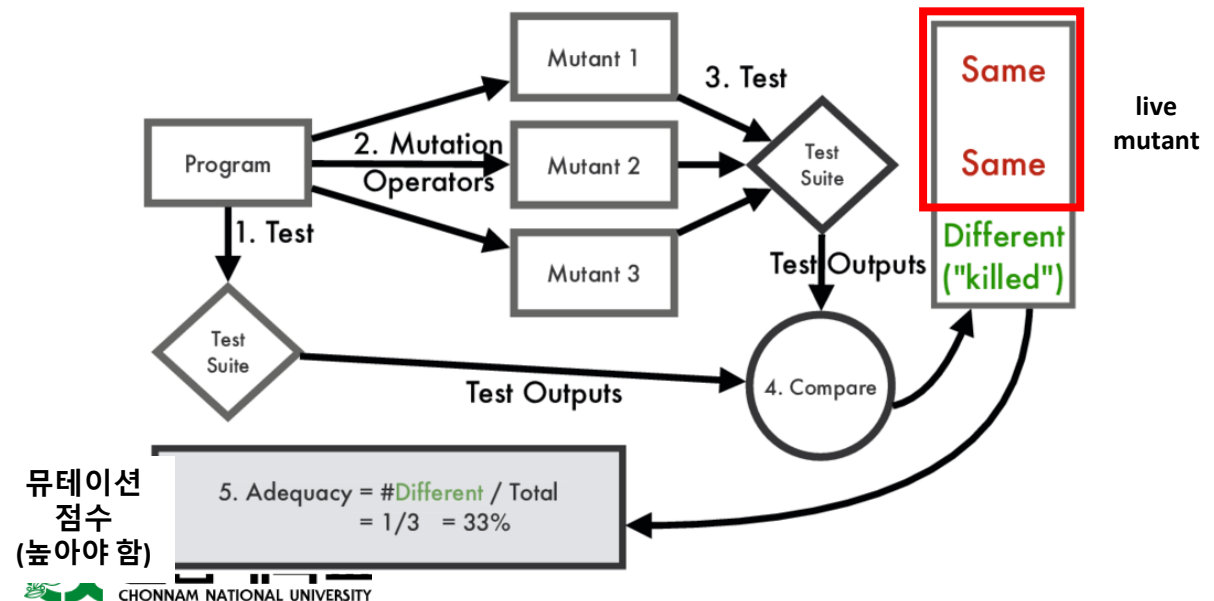
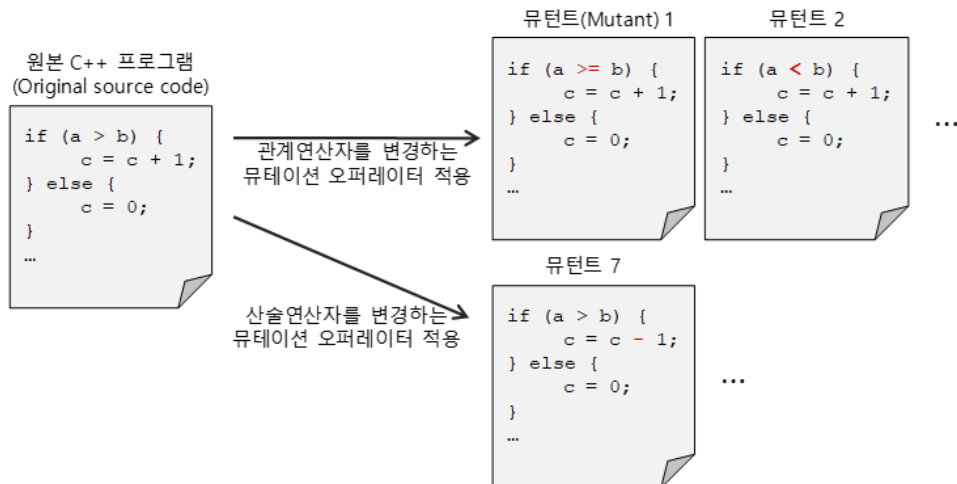
- 배경: 명세 기반 설계 → 블랙박스 테스트 + 구조 기반 설계 → 화이트박스 테스트

▶ 100% 커버하더라도 버그가 있을 수 있음

- 코드를 일부러 변형해 뮤턴트 프로그램을 만들고, 테스트 케이스가 이를 잡는지 확인하는 방법

달라질 수 있도록,
신규 테스트 추가

= 테스트 스위트의 품질 평가



그 외에도...

• 퍼징 테스트

- 프로그램에 랜덤하거나 잘못된 입력값을 넣어서 버그, 크래시, 보안취약점을 찾는 동적 테스트 기법
- 위 목적으로 무작위적인 입력값(Test Case)들을 생성하는 소프트웨어 테스트 기법
 - Input: 무작위 입력 값, Expected Output: 오류 없이 실행
- 가능한 높은 실행 coverage를 달성하면서 많은 버그들을 찾아내는 것이 목표

`<test>`
Lets Fuzz
`</test>`

Bit Flipping

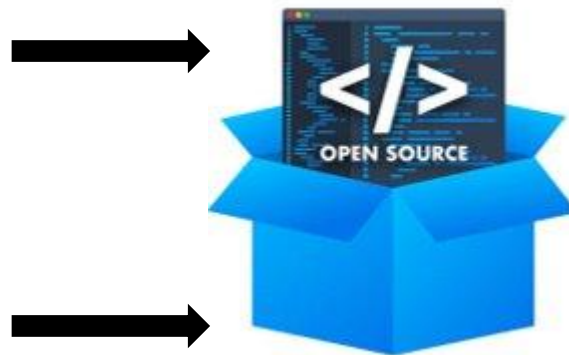
`<tes_>`
L' t\$ F) zZ
`</teSt_>`

Mutation기법 중 하나인 Bit Flipping은 특정 비트를 일정한 주기 또는 무작위로 바꿔버리는 것입니다.

`<test>`
Lets Fuzz
`</test>`

Append Random String

`<test>`
Lets Fuzzzzzzzz
`</test>`



디버깅 필요



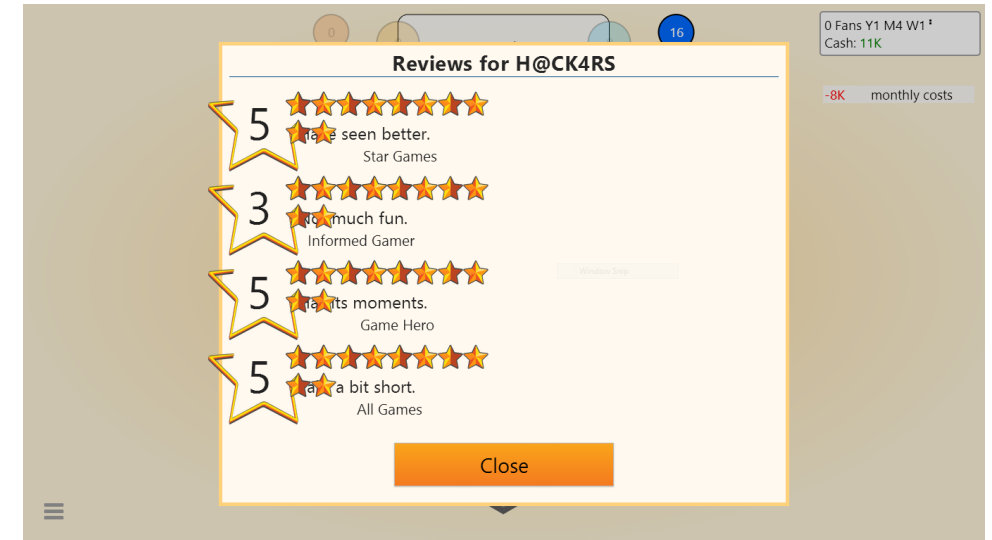
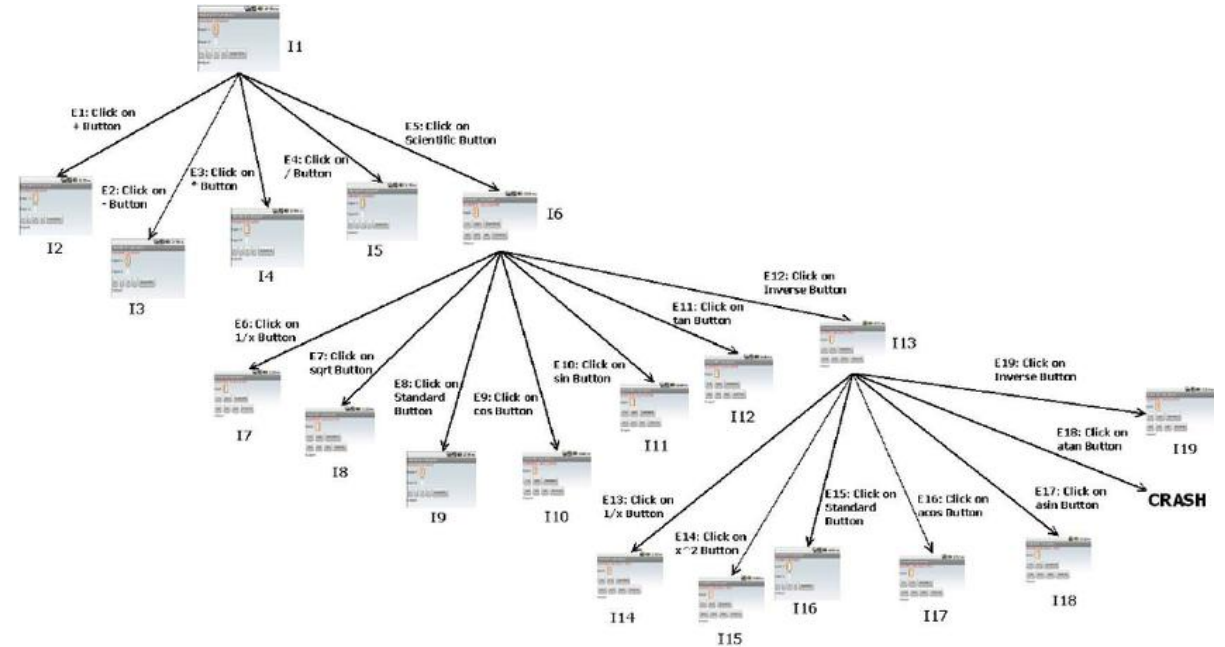
프로그램 바로 종료!

에러 없이 동작

그 외에도...

• GUI 테스트

- 목표: 최대한 많은 화면 확인해보기!
 - 기능적 오류: 버튼 클릭 시 미동작
 - 레이아웃 오류: UI 깨짐, 요소 겹침
 - 입력 처리 오류: validation 실패
 - 상태 관리 오류: 로그인/로그아웃 상태 불일치
 - 호환성 오류: 크롬은 되는데, 사파리 안됨
 - Rendering 오류



그 외에도...

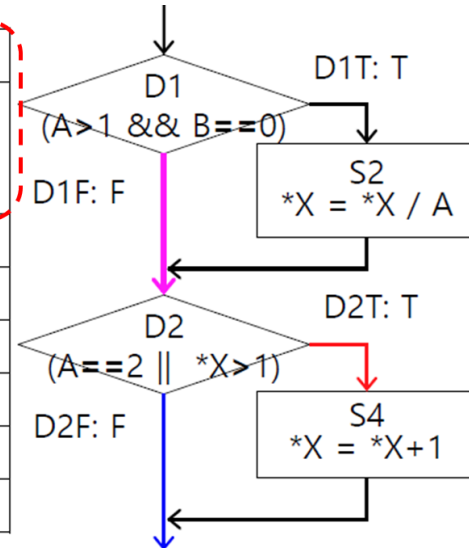
- 속성 기반 테스트
- 뮤테이션 테스트
- 퍼징 테스트
- GUI 테스트
- 메타모픽 테스트
- API/네트워크/서버부하/성능 테스트
- ▶ 이를 자동화하려는 연구도 많음

테스트 케이스 자동 생성의 필요성

- 소프트웨어 규모 증가 → 모든 경로/명세를 기반으로 TC를 사람이 설계하기 어려움
 - 명세 기반: 명세를 작성하고 관리하는데 부담이 큼
 - 구조 기반: 커버리지를 100% 채우기 위한 TC 설계 어렵고, 100% 채워도 버그 못 잡을 수 있음
- ▶ LLM의 언어/코드 이해와 생성 능력을 바탕으로 자동 생성하는 기술/연구 증가

회원 종류	배출비 쿠폰	결제 방법
비회원	쿠폰 있음	신용카드
		무통장입금
		휴대폰
	쿠폰 없음	무통장입금
		휴대폰
		신용카드
일반 회원	쿠폰 있음	휴대폰
		신용카드
		무통장입금
	쿠폰 없음	신용카드
		무통장입금
		휴대폰
VIP 회원	쿠폰 있음	무통장입금
		휴대폰
		신용카드
	쿠폰 없음	무통장입금
		휴대폰
		신용카드

	TC1	TC1		TC1	TC2
	A=1	A=2		A=1	A=2
	B=0	B=1		B=0	B=1
	*X=3	*X=1		*X=3	*X=1
A > 1	F	T	D1T		
B == 0	T	F	D1F	✓	✓
A == 2	F	T	D2T	✓	✓
*X > 1	T	F	D2F		
	4 / 8	4 / 8		2 / 4	2 / 4
	8 / 8			2 / 4	



LLM 기반 테스트 코드 자동 생성

- LLM (대형 언어 모델, Large Language Model)

- 입력 데이터를 이해하고, 그에 따른 결과물을 생성하는 능력 탁월
- CodeLLM: 소스 코드 + 자연어가 학습된 LLM

▶ 테스트 코드를 자동으로 생성해달라고 하면 어떨까?



Large Language Models as Test Case Generators: Performance Evaluation and Enhancement, 2024

- TestChain 제안 = 입출력 단계를 분리

- Designer Agent : 테스트 입력 생성
- Calculator Agent : 테스트 결과(출력) 생성
- (Agent: 인간 대리인, LLM 추론 과정과 도구를 결합해서 작업 자동화)

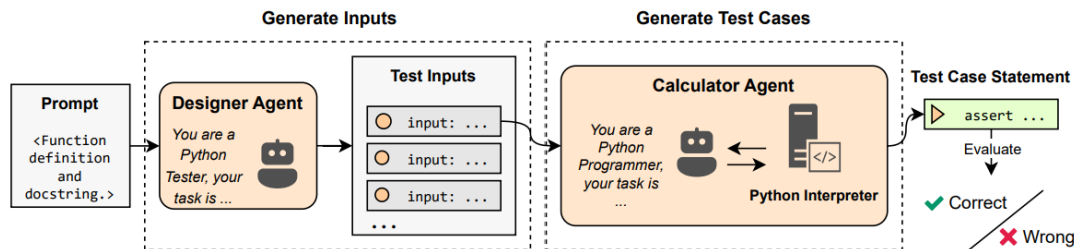


Figure 4: Illustration of the TestChain framework.

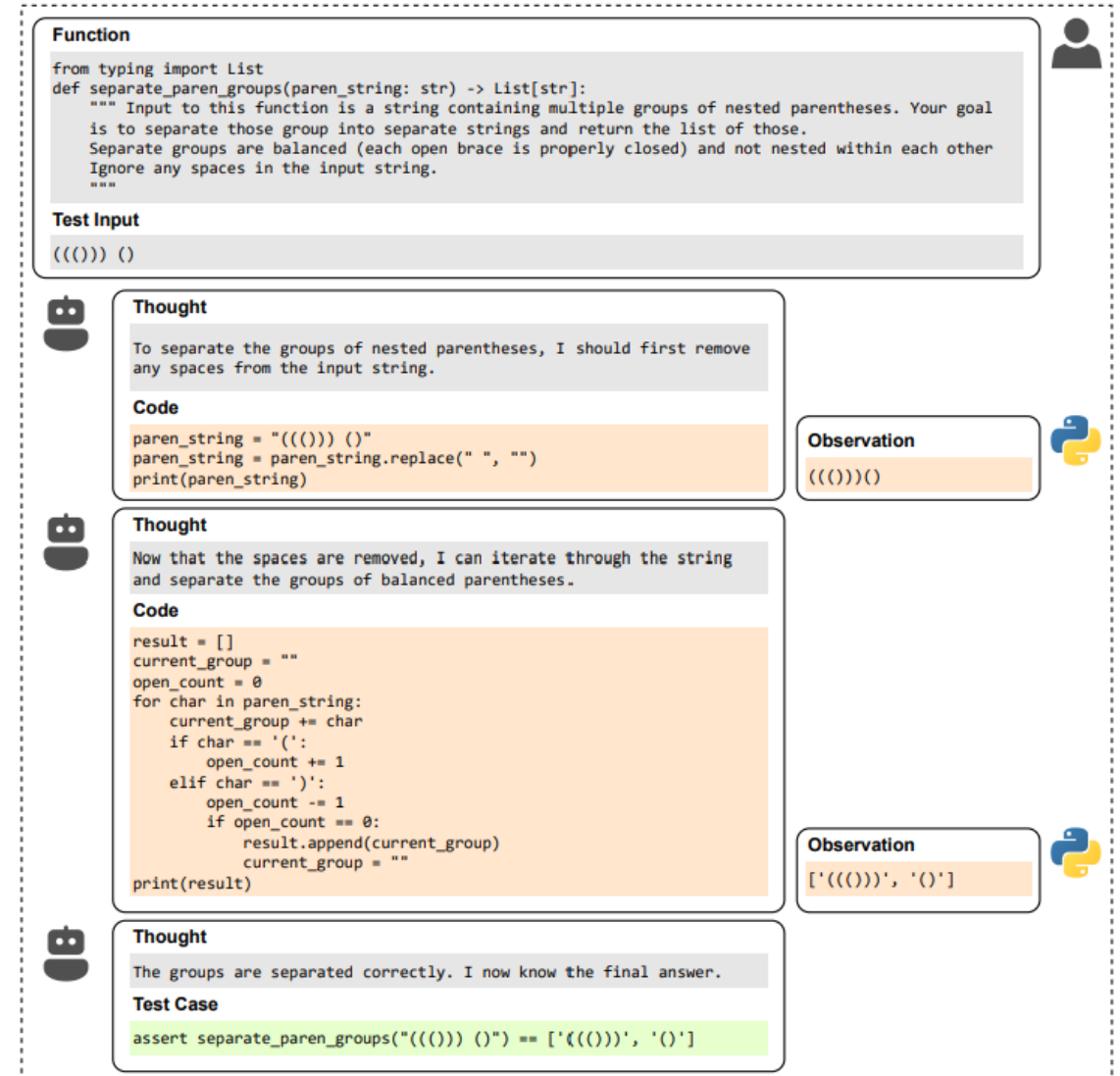


Figure 5: Example of the conversation process produced by Calculator agent.

실습 6. 테스트 코드 작성해보기

(신규 프로젝트로, git/github X)

테스트 코드와 테스트 프레임워크

- 일반적인 코드로 테스트 코드 작성 시 문제

- 출력된 메시지를 확인해야 테스트 성공/실패를 알 수 있음
- 실패했을 때 어디서, 왜 실패 했는지 자동 추적 어려움
- 테스트가 많아지면 코드가 복잡해짐
- 일반적인 프로덕션 코드와 구분이 안되어, 자동 실행이나 보고가 어려움

```
def test_login_success():  
    result = login("user1", "1234")  
    if result == True:  
        print("테스트 통과")  
    else:  
        print("테스트 실패")
```

- ▶ 테스트 관련 라이브러리/프레임워크 사용!

- 테스트를 자동으로 실행
- 결과를 일관된 형식으로 보여줌
- 테스트를 그룹화해서 관리

테스트 프레임워크와 단언문

- 단언문 (assert): 특정 조건이 반드시 참(True) 이어야 한다고 선언하는 문장
 - 프로그램이 기대하는 대로 동작하는지 확인
 - 코드 안에 “이 부분은 이래야 한다”는 계약(contract)를 명시 (협업할 때 코드 자체가 명세처럼 동작)
 - 버그가 발생했을 때, 바로 그 지점을 알 수 있음.

- In 테스트 프레임워크

- 단언문: 테스트 성공/실패를 판정하는 기준
- 프레임워크: 그 기준을 실행/결과수집/리포팅 하는 시스템

```
def test_login_success():  
    assert login("user1", "1234") == True
```

```
def test_login_success():  
    result = login("user1", "1234")  
    if result == True:  
        print("테스트 통과")  
    else:  
        print("테스트 실패")
```



테스트 코드 작성 방법

- Python: unittest 라이브러리 사용

- Python 표준 라이브러리로 제공되는 테스트 자동화 도구 (그 외로, pytest가 있음)
- 자바에서는 junit, C++에서는 GoogleTest가 자주 사용됨
- 기본 개념
 - TestCase : 하나의 테스트 단위를 표현하는 클래스
 - assertEquals(a, b) : $a == b$ 인지 확인하는 테스트 메서드
 - setUp() / tearDown() : 테스트 전/후에 자동으로 실행되는 초기화 코드
 - test_ {함수명} : unittest가 인식하는 테스트 함수

테스트 코드 작성 방법

• 간단한 예시

```
import unittest

# 테스트 대상 코드
class Calculator:
    def add(self, a, b):
        return a + b

# 테스트 코드
class TestCalculator(unittest.TestCase):
    def test_add(self):
        calc = Calculator()
        self.assertEqual(calc.add(2, 3), 5)
        self.assertEqual(calc.add(-1, 1), 0)
        self.assertEqual(calc.add(0, 0), 0)

if __name__ == '__main__':
    unittest.main()
```

```
.
-----
Ran 1 test in 0.000s
OK
```

```
project/
├─ calculator.py      # 프로덕션 코드
└─ test_calculator.py # 테스트 코드
```

```
# calculator.py
class Calculator:
    def add(self, a, b):
        return a + b

# test_calculator.py
import unittest
from calculator import Calculator

class TestCalculator(unittest.TestCase):
    def test_add(self):
        calc = Calculator()
        self.assertEqual(calc.add(2, 5), 5)
        self.assertEqual(calc.add(-1, 0), 0)
        self.assertEqual(calc.add(0, 0), 0)

if __name__ == '__main__':
    unittest.main()
```

테스트 코드 작성 방법

- 간단한 예시: 테스트 코드 에러 (+ 에러 코드 테스트 시)

```
1 import unittest
2
3 # 테스트 대상 코드
4 class Calculator:
5     def add(self, a, b):
6         return a + b
7
8 # 테스트 코드
9 class TestCalculator(unittest.TestCase):
10     def test_add(self):
11         calc = Calculator()
12         self.assertEqual(calc.add(2, 5), 5)
13         self.assertEqual(calc.add(-1, 0), 0)
14         self.assertEqual(calc.add(0, 0), 0)
15
16 if __name__ == '__main__':
17     unittest.main()
```

```
F
=====
FAIL: test_add (__main__.TestCalculator)
-----
Traceback (most recent call last):
  File "main.py", line 12, in test_add
    self.assertEqual(calc.add(2, 5), 5)
AssertionError: 7 != 5
-----
Ran 1 test in 0.000s
FAILED (failures=1)
```

```
1 import unittest
2
3 # 테스트 대상 코드
4 class Calculator:
5     def add(self, a, b):
6         return a + b
7
8 # 테스트 코드
9 class TestCalculator(unittest.TestCase):
10     def test_add(self):
11         calc = Calculator()
12         self.assertEqual(calc.add(2, 3), 5)
13         self.assertEqual(calc.add(-1, 0), 0)
14         self.assertEqual(calc.add(0, 0), 0)
15
16 if __name__ == '__main__':
17     unittest.main()
```

```
F
=====
FAIL: test_add (__main__.TestCalculator)
-----
Traceback (most recent call last):
  File "main.py", line 13, in test_add
    self.assertEqual(calc.add(-1, 0), 0)
AssertionError: -1 != 0
-----
Ran 1 test in 0.000s
```

테스트 코드 작성 방법

- test_ {함수명} 으로 안주면 test code로 인식 안함

```
1 import unittest
2
3 class MyTest(unittest.TestCase):
4     def test_add(self):
5         self.assertEqual(1 + 1, 2)
6
7     def test_add2(self):
8         self.assertEqual(1 + 1, 2)
9
10 if __name__ == '__main__':
11     unittest.main()
```

•
•

Ran 2 tests in 0.000s

OK

```
1 import unittest
2
3 class MyTest(unittest.TestCase):
4     def test_add(self):
5         self.assertEqual(1 + 1, 2)
6
7     def calc_add(self):
8         self.assertEqual(1 + 1, 2)
9
10 if __name__ == '__main__':
11     unittest.main()
12
```

•

Ran 1 test in 0.000s

OK

테스트 코드 작성 방법

- Setup과 teardown

- 단일 테스트 마다 준비/종료 과정이 같을 때 유용

```
1 import unittest
2
3 # 프로덕션 코드
4 class Calculator:
5     def add(self, a, b):
6         return a + b
7
8     def subtract(self, a, b):
9         return a - b
10
```

```
11 # 테스트 코드
12 class TestCalculator(unittest.TestCase):
13
14     def setUp(self):
15         print(">> setUp 실행됨")
16         self.calc = Calculator() # 모든 테스트 전에 새로운 객체 생성
17
18     def tearDown(self):
19         print("<< tearDown 실행됨")
20
21     def test_add(self):
22         self.assertEqual(self.calc.add(2, 3), 5)
23
24     def test_subtract(self):
25         self.assertEqual(self.calc.subtract(5, 3), 2)
26
27 if __name__ == '__main__':
28     unittest.main()
```

```
.
.
-----
Ran 2 tests in 0.004s
OK
>> setUp 실행됨
<< tearDown 실행됨
>> setUp 실행됨
<< tearDown 실행됨
```

테스트 코드 작성 방법

• TestSuite의 필요성

- 테스트할 기능이 많아지면? → 테스트 클래스가 많아질 수 있음. 각 테스트 파일 매번 실행해야 함
 - Logger, Calculator → LoggerTest, CalculatorTest

▶ 어떻게 한 번에 실행시킬까??

```
1 import unittest
2
3 c 2 1 import unittest
4 3 cl 2
5 4 3 class MyTest(unittest.TestCase):
6 5 4 def test_add(self):
7 6 5 | self.assertEqual(1 + 1, 2)
8 7 6
9 8 7 def calc_add(self):
10 9 8 | self.assertEqual(1 + 1, 2)
11 10 i 9
12 11 10 if __name__ == '__main__':
13 12 11 | unittest.main()
14 13 12
```


테스트 코드 작성 방법

• TestSuite 예시

- unittest.TextTestRunner(): 결과를 텍스트로 콘솔에 출력

```
project/  
├─ calculator.py  
├─ test_add.py  
├─ test_subtract.py  
└─ run_all_tests.py
```

```
# calculator.py  
class Calculator:  
    def add(self, a, b):  
        return a + b  
  
    def subtract(self, a, b):  
        return a - b
```

```
# test_add.py  
import unittest  
from calculator import Calculator  
  
class TestAdd(unittest.TestCase):  
    def test_add_positive(self):  
        self.assertEqual(Calculator().add(2, 3), 5)  
  
    def test_add_zero(self):  
        self.assertEqual(Calculator().add(0, 0), 0)
```

```
# test_subtract.py  
import unittest  
from calculator import Calculator  
  
class TestSubtract(unittest.TestCase):  
    def test_subtract_positive(self):  
        self.assertEqual(Calculator().subtract(5, 3), 2)  
  
    def test_subtract_negative(self):  
        self.assertEqual(Calculator().subtract(3, 5), -2)
```

```
# run_all_tests.py  
import unittest  
from test_add import TestAdd  
from test_subtract import TestSubtract
```

```
# 테스트 스위트 생성  
suite = unittest.TestSuite()
```

```
# 테스트 클래스 전체 추가  
loader = unittest.TestLoader()  
suite.addTest(loader.loadTestsFromTestCase(TestAdd))  
suite.addTest(loader.loadTestsFromTestCase(TestSubtract))
```

```
# 테스트 실행기 생성  
runner = unittest.TextTestRunner()  
runner.run(suite)
```

```
....  
-----  
Ran 4 tests in 0.001s  
OK
```

```
# run_selected_tests.py  
import unittest  
from test_add import TestAdd  
from test_subtract import TestSubtract
```

```
# 1. 테스트 스위트 생성  
suite = unittest.TestSuite()
```

```
# 2. 특정 테스트 메서드만 추가 (이름 문자열로 지정)  
suite.addTest(TestAdd('test_add_positive'))  
suite.addTest(TestSubtract('test_subtract_negative'))
```

```
# 3. 실행기 생성 및 실행  
runner = unittest.TextTestRunner()  
runner.run(suite)
```

```
.  
.  
-----  
Ran 2 tests in 0.076s  
OK
```

테스트 코드 평가

- 내가 만든 테스트 케이스(코드들이 내 프로그램을 충분히 테스트 하고 있는가?)

→ 커버리지 평가 필요

- 주로 초록색이 지나간 곳 = 커버한 곳, 붉은색이 안지나간 곳 = 커버 못한 곳
- ► 테스트 스위트가 최대한 모든 코드들을 최소 한번씩을 지나가야 함

```
static bool app_create(void *user_data)
{
    data_initialize();
    data_set_display_change_cb(_display_changed_cb);

    view_set_callbacks(_get_display_string_cb, _key_pressed_cb);

    if (!view_create(NULL))
        return false;

    _set_region_format(); /* This is called to set valid decimal character on the keypad. */

    return true;
}
```

테스트 커버리지 보기!

- coverage 패키지 사용
 - 설치: pip install coverage

calculator.py
test_calculator.py
test_calculator2.py

```
# calculator.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def divide(a, b):
    if b == 0:
        return None
    return a / b
```

```
# test_calculator.py
import unittest
import calculator

class TestCalculator(unittest.TestCase):
    def test_add(self):
        self.assertEqual(calculator.add(2, 3), 5)

    def test_divide(self):
        self.assertEqual(calculator.divide(10, 2), 5)

if __name__ == '__main__':
    unittest.main()
```

```
# test_calculator2.py
import unittest
import calculator

class TestCalculator2(unittest.TestCase):
    def test_subtract(self):
        self.assertEqual(calculator.subtract(5, 2), 3)

if __name__ == '__main__':
    unittest.main()
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\25-1학기\개발론\샘플코드>coverage run -m unittest test_calculator.py
...
Ran 2 tests in 0.001s
OK
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\25-1학기\개발론\샘플코드>coverage report
```

Name	Stmts	Miss	Cover
calculator.py	8	2	75%
test_calculator.py	9	1	89%
TOTAL	17	3	82%

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\25-1학기\개발론\샘플코드>coverage run -m unittest discover
...
Ran 3 tests in 0.002s
OK
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\25-1학기\개발론\샘플코드>coverage report
```

Name	Stmts	Miss	Cover
calculator.py	8	1	88%
test_calculator2.py	7	1	86%
test_calculator.py	9	1	89%
TOTAL	24	3	88%

테스트 커버리지 보기!

- coverage 패키지 사용

- 설치: pip install coverage

calculator.py

test_calculator.py

test_calculator2.py

```
1 # calculator.py
2 def add(a, b):
3     return a + b
4
5 def subtract(a, b):
6     return a - b
7
8 def divide(a, b):
9     if b == 0:
10         return None
11     return a / b
```

```
1 # test_calculator.py
2 import unittest
3 import calculator
4
5 class TestCalculator(unittest.TestCase):
6     def test_add(self):
7         self.assertEqual(calculator.add(2, 3), 5)
8
9     def test_divide(self):
10         self.assertEqual(calculator.divide(10, 2), 5)
11
12 if __name__ == '__main__':
13     unittest.main()
```

```
1 # test_calculator2.py
2 import unittest
3 import calculator
4
5 class TestCalculator2(unittest.TestCase):
6     def test_subtract(self):
7         self.assertEqual(calculator.subtract(5, 2), 3)
8
9 if __name__ == '__main__':
10     unittest.main()
```

```
def test_divide(self):
    self.assertEqual(calculator.divide(2, 0), 0)
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\25-1학기\개발론\샘플코드>coverage run -m unittest discover
.F.
=====
FAIL: test_divide (test_calculator2.TestCalculator2.test_divide)
-----
Traceback (most recent call last):
  File "C:\Users\user\OneDrive\바탕 화면\강의\25-1학기\개발론\샘플코드\test_calculator2.py", line 9, in test_divide
    self.assertEqual(calculator.divide(2, 0), 0)
AssertionError: None != 0
=====
Ran 4 tests in 0.002s
FAILED (failures=1)
```

```
def test_divide(self):
    self.assertEqual(calculator.divide(2, 0), None)
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\25-1학기\개발론\샘플코드>coverage run -m unittest discover
....
Ran 4 tests in 0.001s
OK
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\25-1학기\개발론\샘플코드>coverage report
Name                               Stmts  Miss  Cover
-----
calculator.py                       8      0   100%
test_calculator2.py                 9      1    89%
test_calculator.py                  9      1    89%
TOTAL                               26      2    92%
```

Name	Stmts	Miss	Cover
calculator.py	8	1	88%
test_calculator2.py	7	1	86%
test_calculator.py	9	1	89%
TOTAL	24	3	88%

실습 7. Mock

(신규 프로젝트로, git/github X)

테스트 코드 작성 방법

- Mock과 Stub = 테스트 시 사용하는 “가짜 기능”

- 기능 테스트를 위해 필요한 외부 시스템이 너무 느리거나, 항상 쓸 수 없을 수 있음
 - 네트워크, DB, 이메일 API, 유료 API 등등..
- 진짜 기능을 사용하지 않고, “가짜 기능”을 만들어서 진짜 기능 대신 실행하게 하자

▶ Mock과 Stub

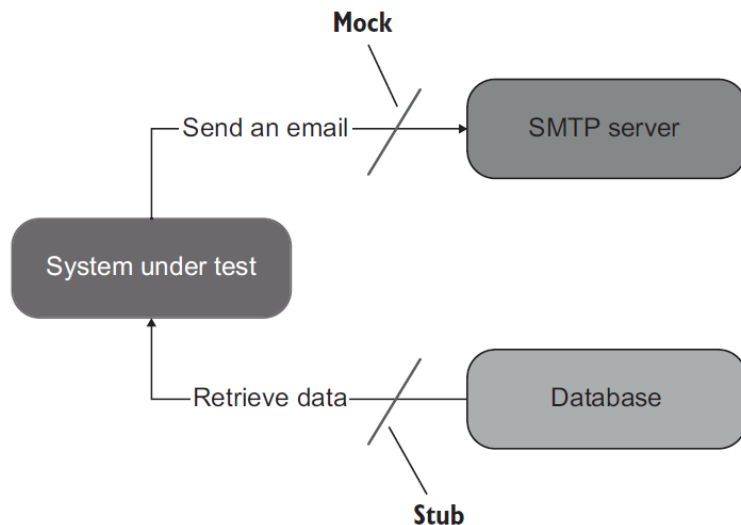
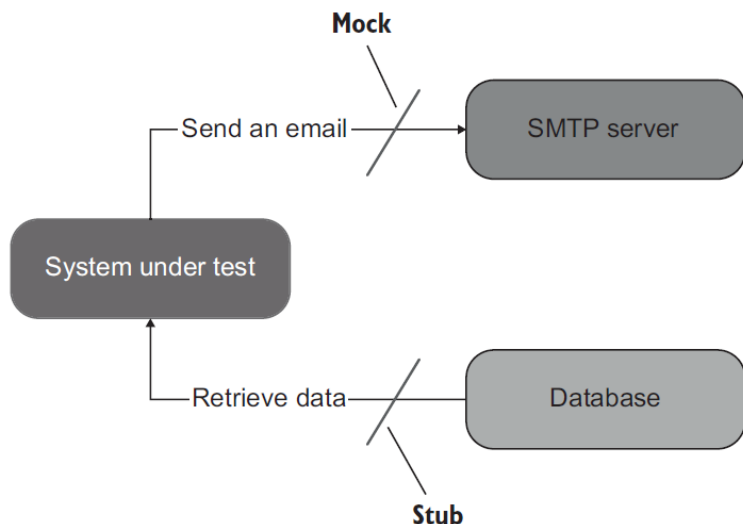


Figure 5.2 Sending an email is an *outcoming* interaction: an interaction that results in a side effect in the SMTP server. A test double emulating such an interaction is a *mock*. Retrieving data from the database is an *incoming* interaction; it doesn't result in a side effect. The corresponding test double is a *stub*.

테스트 코드 작성 방법

- Stub: 외부 의존 기능의 결과를 미리 정해진 값으로 대신하는 가짜 객체



```
1 import unittest
2
3 def get_discount_func(user_id):
4     # 웹 API를 사용해서 가져오는 코드
5     # 시간이 오래 걸린다면?
6
7 def stub_get_discount(user_id):
8     return 1000
9
10 def calculate_price(user_id, price, get_discount_func):
11     return price - get_discount_func(user_id)
12
13 class TestCalculatePrice(unittest.TestCase):
14     def test_discount_applied(self):
15         result = calculate_price("user1", 5000, stub_get_discount)
16         self.assertEqual(result, 4000)
17
18 if __name__ == '__main__':
19     unittest.main()
```

테스트 코드 작성 방법

- Mock: 외부 객체의 행동을 흉내내면서, 호출 여부와 어떤 값으로 호출됐는지 추적
 - Stub: 결과를 미리 정해진 값으로 대신만 함!

```
1 # email_sender.py
2 class EmailSender:
3     def send(self, email):
4         print(f"Sending email to {email}")
5
```

```
1 # user_service.py
2 class UserService:
3     def __init__(self, email_sender):
4         self.email_sender = email_sender
5
6     def register(self, email):
7         self.email_sender.send(email)
8         return "Registered"
```

```
1 # test_user_service.py
2 import unittest
3 from unittest.mock import Mock
4 from user_service import UserService
5
6 class TestUserService(unittest.TestCase):
7     def test_register_sends_email(self):
8         # 1. Mock 객체 생성
9         mock_sender = Mock()
10
11         # 2. Mock을 의존성으로 주입
12         service = UserService(mock_sender)
13
14         # 3. 기능 실행
15         result = service.register("test@example.com")
16
17         # 4. 결과값 확인
18         self.assertEqual(result, "Registered")
19
20         # 5. 이메일 전송 함수가 **정확히 호출되었는지 확인**
21         mock_sender.send.assert_called_with("test@example.com")
22
23 if __name__ == '__main__':
24     unittest.main()
```


테스트 코드 작성 방법

- Mock: 외부 객체의 행동을 흉내내면서, 호출 여부와 어떤 값으로 호출됐는지 추적
 - Stub: 결과를 미리 정해진 값으로 대신만 함!
 - Mock을 Stub처럼 사용하기

```
1 import unittest
2
3 def get_discount_func(user_id):
4     # 웹 API를 사용해서 가져오는 코드
5     # 시간이 오래 걸린다면
6
7     def stub_get_discount(user_id):
8         return 1000
9
10 def calculate_price(user_id, price, get_discount_func):
11     return price - get_discount_func(user_id)
12
13 class TestCalculatePrice(unittest.TestCase):
14     def test_discount_applied(self):
15         result = calculate_price("user1", 5000, stub_get_discount)
16         self.assertEqual(result, 4000)
17
18 if __name__ == '__main__':
19     unittest.main()
```

```
1 from unittest.mock import Mock
2 import unittest
3
4 def calculate_price(user_id, price, get_discount_func):
5     return price - get_discount_func(user_id)
6
7 class TestCalculatePrice(unittest.TestCase):
8     def test_discount_applied_with_mock_as_stub(self):
9         mock_func = Mock()
10        mock_func.return_value = 2000 # Stub처럼 리턴값 고정
11
12        result = calculate_price("user123", 8000, mock_func)
13        self.assertEqual(result, 6000)
14
15 if __name__ == '__main__':
16     unittest.main()
```

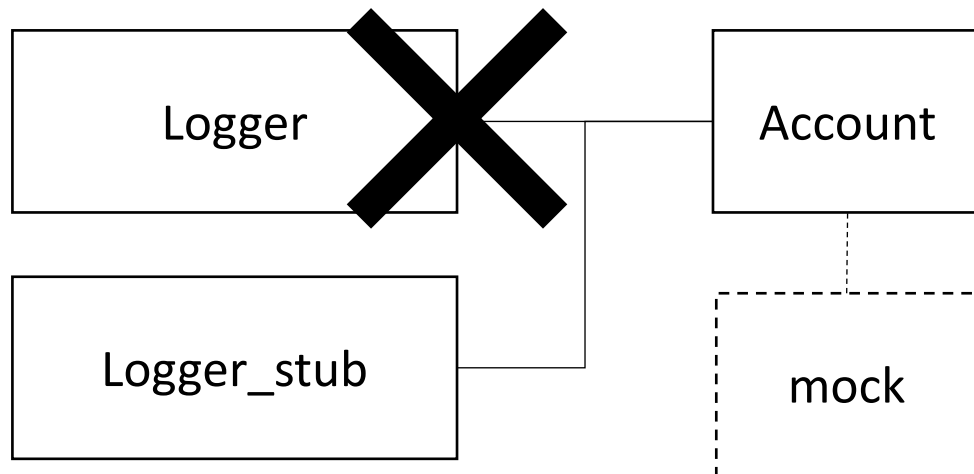
실습-은행 시스템의 계좌(Account) 클래스 테스트

- 요구사항

- 입금(deposit)과 출금(withdraw) 기능 제공
- 출금 시 잔액이 부족하면 InsufficientFundsError 예외 발생
- 계좌 거래 내역은 Logger를 통해 기록

- 현재 상황: Logger는 개발되지 않음

- 현재 상태에서의 account.py의 테스트 필요



```
# account.py
# 사용자 정의 예외
class InsufficientFundsError(Exception):
    pass

# 계좌 클래스
class Account:
    def __init__(self, logger):
        self.balance = 0
        self.logger = logger # 거래 기록기 (개발 안됨)

    def deposit(self, amount):
        self.balance += amount
        self.logger.log(f"Deposited {amount}")
        return self.balance

    def withdraw(self, amount):
        if self.balance < amount:
            self.logger.log("Withdrawal failed: insufficient funds")
            raise InsufficientFundsError("잔액 부족")
        self.balance -= amount
        self.logger.log(f"Withdrew {amount}")
        return self.balance
```

실습-은행 시스템의 계좌(Account) 클래스 테스트

logger_stub.py

```
class StubLogger:
    def log(self, message):
        # 단순히 항상 "logged" 문자열 반환 (실제 로깅은 아님)
        return "logged"
```

test_account_stub.py

```
import unittest
from account import Account, InsufficientFundsError
from logger_stub import StubLogger

class TestAccountWithStub(unittest.TestCase):
    def setUp(self):
        self.logger = StubLogger()
        self.account = Account(self.logger)

    def test_deposit_increases_balance(self):
        self.account.deposit(100)
        self.assertEqual(self.account.balance, 100)

    def test_withdraw_raises_error_on_insufficient_funds(self):
        with self.assertRaises(InsufficientFundsError):
            self.account.withdraw(50)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

account.py

사용자 정의 예외

```
class InsufficientFundsError(Exception):
    pass
```

계좌 클래스

```
class Account:
    def __init__(self, logger):
        self.balance = 0
        self.logger = logger # 거래 기록기 (개발 안됨)

    def deposit(self, amount):
        self.balance += amount
        self.logger.log(f"Deposited {amount}")
        return self.balance

    def withdraw(self, amount):
        if self.balance < amount:
            self.logger.log("Withdrawal failed: insufficient funds")
            raise InsufficientFundsError("잔액 부족")
        self.balance -= amount
        self.logger.log(f"Withdrew {amount}")
        return self.balance
```

실습-은행 시스템의 계좌(Account) 클래스 테스트

- Stub과 mock 중 하나를 선택해서 테스트

- 주로 Mock을 사용!

test_account_mock.py

```
import unittest
from unittest.mock import Mock
from account import Account
```

```
class TestAccountWithMock(unittest.TestCase):
    def setUp(self):
```

```
        self.mock_logger = Mock() # Mock으로 대체
        self.account = Account(self.mock_logger)
```

```
    def test_deposit_calls_logger(self):
        self.account.deposit(200)
        # assert_called_with(인자) = 마지막 호출 시 인자와 동일한지 확인
        self.mock_logger.log.assert_called_with("Deposited 200")
```

```
    def test_withdraw_calls_logger(self):
        self.account.deposit(100)
        self.account.withdraw(50)
        # assert_any_call(인자) = 여러 번 호출됐을 때, 호출 내역 중 인자가 일치하는지
        self.mock_logger.log.assert_any_call("Withdrew 50")
```

```
if __name__ == "__main__":
    unittest.main(verbosity=2)
```

mock_logger에 log() 자동으로 생성됨

account.py

사용자 정의 예외

```
class InsufficientFundsError(Exception):
    pass
```

계좌 클래스

```
class Account:
    def __init__(self, logger):
        self.balance = 0
        self.logger = logger # 거래 기록기 (개발 안됨)
```

```
    def deposit(self, amount):
        self.balance += amount
        self.logger.log(f"Deposited {amount}")
        return self.balance
```

```
    def withdraw(self, amount):
        if self.balance < amount:
            self.logger.log("Withdrawal failed: insufficient funds")
            raise InsufficientFundsError("잔액 부족")
        self.balance -= amount
        self.logger.log(f"Withdrew {amount}")
        return self.balance
```

실습-은행 시스템의 계좌(Account) 클래스 테스트

- 일반적인 프로젝트 구조로 개선

- src: 프로덕션 코드
- tests: 테스트 코드

기존	개선
account.py test_mock.py logger_stub.py test_stub.py	src - account.py tests - test_mock.py

- 전체 테스트 자동 실행

- 루트 디렉토리에서 아래 명령어 실행
- Python -m unittest discover -s tests -v
 - -m : python 모듈을 스크립트처럼 실행 (unittest)
 - discover : 테스트 자동 검색 모드
 - -s tests : 검색 시작 (start) 폴더 설정 (tests 폴더)
 - -v : verbose 모드 (테스트 이름 출력)

```
test_deposit_calls_logger (test_mock.TestAccountWithMock.test_deposit_calls_logger) ... ok
test_withdraw_calls_logger (test_mock.TestAccountWithMock.test_withdraw_calls_logger) ... ok

-----
Ran 2 tests in 0.001s

OK
```

실습-은행 시스템의 계좌(Account) 클래스 테스트

- coverage 확인

- coverage run -m unittest discover -s tests
- coverage report -m
 - m missing line 정보 요약

Name	Stmts	Miss	Cover	Missing
src\account.py	17	2	88%	19-20
tests\test_mock.py	16	1	94%	21

TOTAL	33	3	91%	

- coverage html

Coverage report: 91%

☐ hide covered

coverage.py v7.10.6, created at 2025-09-22 18:18 +0900

File ▲	statements	missing	excluded	coverage
src\account.py	17	2	0	88%
tests\test_mock.py	16	1	0	94%
Total	33	3	0	91%

Coverage for **src\account.py**: 88%

17 statements 15 run 2 missing 0 excluded

« prev ^ index » next coverage.py v7.10.6, created at 2025-09-22 18:18 +0900

```
1
2 # 사용자 정의 예외
3 class InsufficientFundsError(Exception):
4     pass
5
6 # 계좌 클래스
7 class Account:
8     def __init__(self, logger):
9         self.balance = 0
10        self.logger = logger # 거래 기록기 (stub/mock 주입)
11
12    def deposit(self, amount):
13        self.balance += amount
14        self.logger.log(f"Deposited {amount}")
15        return self.balance
16
17    def withdraw(self, amount):
18        if self.balance < amount:
19            self.logger.log("Withdrawal failed: insufficient funds")
20            raise InsufficientFundsError("잔액 부족")
21        self.balance -= amount
22        self.logger.log(f"Withdrew {amount}")
23        return self.balance
```

실습 8. 간단한 속성 기반 테스트 (TC 자동생성)

Simple Fuzzing

- pip install hypothesis
- python fuzz_json.py
- pytest -q fuzz_json.py

```
Traceback (most recent call last):
  File "c:\Users\user\Desktop\강의자료\검증\실습\fuzz_hypothesis_json.py", line 10, in <module>
    test_process_input() # Hypothesis가 자동으로 여러 사례를 생성하
    ~~~~~
  File "c:\Users\user\Desktop\강의자료\검증\실습\fuzz_hypothesis_json.py", line 10, in test_process_input
    def test_process_input(s):
    ~~~~~
  File "C:\Users\user\anaconda3\Lib\site-packages\hypothesis\core.py", line 1000, in raise_the_error_hypothesis_found
    raise the_error_hypothesis_found
  File "c:\Users\user\Desktop\강의자료\검증\실습\fuzz_hypothesis_json.py", line 10, in process_input
    process_input(s)
    ~~~~~
  File "c:\Users\user\Desktop\강의자료\검증\실습\fuzz_hypothesis_json.py", line 10, in if obj.get("action") == "divide":
    if obj.get("action") == "divide":
    ~~~~~
AttributeError: 'float' object has no attribute 'get'
Falsifying example: test_process_input(
  s='NaN',
)
Explanation:
  These lines were always and only run by failing examples:
  C:\Users\user\anaconda3\Lib\json\decoder.py:349
```

```
fuzz_hypothesis_json.py:22: in test_process_input
    process_input(s)
    ~~~~~
s = 'NaN'

def process_input(s: str):
    # 실제 코드에서는 여기서 더 복잡한 처리(데이터베이스, 파일쓰기 등)가 들어갈 수 있음
    obj = json.loads(s) # -> 여기서 취약점(예외 발생) 지점이 될 수 있음
    # if "action" not in obj:
    #     raise ValueError("no action")
    if obj.get("action") == "divide":
    # 잘못된 타입으로 인해 ZeroDivisionError 등 발생 가능
    a = obj.get("a", 1)
    b = obj.get("b", 1)
    return a / b
    ~~~~~
>
E   AttributeError: 'float' object has no attribute 'get'
E   Falsifying example: test_process_input(
E       s='NaN',
E   )

fuzz_hypothesis_json.py:10: AttributeError
NaN
NaN
=====
FAILED fuzz_hypothesis_json.py::test_process_input - AttributeError: '
=====
```

```
# fuzz_json.py
from hypothesis import given, strategies as st
import json
```

테스트할 대상 함수 (예: 외부 입력을 JSON으로 파싱하고 특정 키를 기대)

```
def process_input(s: str):
    # 실제 코드에서는 여기서 더 복잡한 처리(데이터베이스, 파일쓰기 등)가 들어갈 수 있음
    obj = json.loads(s) # -> 여기서 취약점(예외 발생) 지점이 될 수 있음
    # if "action" not in obj:
    #     raise ValueError("no action")
    if obj.get("action") == "divide":
    # 잘못된 타입으로 인해 ZeroDivisionError 등 발생 가능
    a = obj.get("a", 1)
    b = obj.get("b", 1)
    return a / b
    ~~~~~
return obj
```

Hypothesis가 다양한 문자열을 생성해서 process_input에 넣음
@given(st.text())

```
def test_process_input(s):
    try:
        print(s)
        process_input(s)
    except (json.JSONDecodeError, ValueError, ZeroDivisionError):
    # 예외 자체를 실패로 취급하지 않고, 어떤 예외가 나는지 확인하는 용도
    # 실제 퍼징 목적: 예외가 발생했을 때 원치 않는 크래시(예: interpreter crash)가 있는지 확인
    pass
```

```
if __name__ == "__main__":
    test_process_input() # Hypothesis가 자동으로 여러 사례를 생성하여 실행
```


Simple Fuzzing

- pip install hypothesis
- python fuzz_gaussian.py
- pytest -q fuzz_gaussian.py

```
| ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
+----- 1 -----
Traceback (most recent call last):
  File "c:\Users\user\Desktop\강의자료\검증\실습\hypothesis_gaussian.py", line 11, in <module>
    out = gaussian(x, mu, sigma)
  File "c:\Users\user\Desktop\강의자료\검증\실습\gaussian.py", line 5, in gaussian
    return 1 / sqrt(2 * pi * sigma**2) * exp(-((x - mu) ** 2) / (2 * sigma**2))
OverflowError: (34, 'Result too large')
Falsifying example: test_gaussian_simple(
  x=0.0,
  mu=0.0,
  sigma=1.3407807929942597e+154,
)
+----- 2 -----
Traceback (most recent call last):
  File "c:\Users\user\Desktop\강의자료\검증\실습\hypothesis_gaussian.py", line 11, in <module>
    out = gaussian(x, mu, sigma)
  File "c:\Users\user\Desktop\강의자료\검증\실습\gaussian.py", line 5, in gaussian
    return 1 / sqrt(2 * pi * sigma**2) * exp(-((x - mu) ** 2) / (2 * sigma**2))
ZeroDivisionError: float division by zero
Falsifying example: test_gaussian_simple(
  x=0.0,
  mu=0.0,
  sigma=0.0,
)
```

```
=====
EXCEPTION: ZeroDivisionError float division by zero   input: 0.0 0.0 0.0
EXCEPTION: OverflowError (34, 'Result too large')    input: 0.0 0.0 1.3407807929942
EXCEPTION: OverflowError (34, 'Result too large')    input: 0.0 0.0 1.3407807929942
EXCEPTION: ZeroDivisionError float division by zero   input: 0.0 0.0 0.0
=====
fuzz_hypothesis_gaussian.py::test_gaussian_simple
fuzz_hypothesis_gaussian.py::test_gaussian_simple
C:\Users\user\Desktop\강의자료\검증\실습\gaussian.py:5: RuntimeWarning: divide b
  return 1 / sqrt(2 * pi * sigma**2) * exp(-((x - mu) ** 2) / (2 * sigma**2))
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=====
FAILED fuzz_hypothesis_gaussian.py::test_gaussian_simple - ExceptionGroup: Hypothe
```

```
# gaussian.py
from numpy import exp, pi, sqrt

def gaussian(x, mu: float = 0.0, sigma: float = 1.0) -> float:
    return 1 / sqrt(2 * pi * sigma**2) * exp(-((x - mu) ** 2) / (2 * sigma**2))
```

```
# fuzz_gaussian.py
from hypothesis import given, strategies as st, settings
import importlib, math, traceback
from gaussian import gaussian
```

```
def is_finite(v):
    try:
        return not (math.isnan(v) or math.isinf(v))
    except Exception:
        return False
```

```
@settings(max_examples=1000, deadline=None)
```

```
@given(
    x=st.floats(allow_nan=True, allow_infinity=True),
    mu=st.floats(allow_nan=True, allow_infinity=True),
    sigma=st.floats(allow_nan=True, allow_infinity=True),
)
```

```
def test_gaussian_simple(x, mu, sigma):
    try:
        # print(x, mu, sigma)
        out = gaussian(x, mu, sigma)
    except Exception as e:
        print("EXCEPTION:", type(e).__name__, e, " input:", x, mu, sigma)
        # re-raise so Hypothesis will try to shrink and show reproduce info
        raise
```

```
if not is_finite(out):
    print("NON-FINITE OUTPUT:", out, " input:", x, mu, sigma)
    # make test fail to get Hypothesis shrinking / reproduce info
    assert False, f"Non-finite output {out} for input {x}, {mu}, {sigma}"
```

```
if __name__ == "__main__":
    # Run the Hypothesis test directly
    test_gaussian_simple()
    print("Done (no issues found in sampled cases).")
```

숙제 4. 실습 6~8

- 이번 주 일요일 자정까지

- 자유 양식

1. 실습 6

- 목적, 방법, 결과 스크린샷 및 설명

2. 실습 7

- 목적, 방법, 결과 스크린샷 및 설명

3. 실습 8

- 목적, 방법, 결과 스크린샷 및 설명

- 이력은 확인 안 하지만, 꼭 해보세요